Declared Ambiguity: The Agent Execution Protocol (AEP) for Autonomous Agents Calling Non-Idempotent Legacy APIs

Anonymous Author(s)

***Abstract*—Autonomous software agents increasingly invoke legacy enterprise APIs that are non-idempotent, accept no idempotency key, and cannot be asked, after the fact, whether a mutation was applied. When an agent crashes around such a call, every available strategy loses something. Retrying risks a second real-world effect that nobody observes. Not retrying risks an effect that exists and that no record accounts for. We argue that the choice between these is not a question of engineering quality but a three-way trade, and that a system's obligation is to make the third corner reachable: to convert silent failure into *declared, durable, bounded ambiguity* that an operator can act on. We present the Agent Execution Protocol (AEP), a fail-closed execution protocol in which every external side effect is preceded by a durably acknowledged write-ahead intent, every state write is fenced by lock ownership and an exact expected-version compare-and-swap in one atomic script, and every unresolvable outcome halts and escalates rather than guessing. We evaluate AEP against five baseline designs, one of them in two configurations — naive retry, lease-only, CAS-only, an ablation without the durability barrier, and a durable-execution engine in both its vendor-default and at-most-once settings — under real `SIGKILL` process faults, across three endpoint reconciliation capabilities. Two narrower faults target the barrier and its premise: a hard Redis kill, which we use as an AEP-versus-ablation comparison on one capability class, and block-level write loss, which we use to test that premise on durable records rather than on the protocol's outcomes.**

Our central result separates two claims that the write-ahead pattern is usually sold as one. *Detection* — no undetected duplicate and no lost effect, with a residual of declared ambiguity whose rate is set by what the endpoint can be asked — is produced by the pre-dispatch record and a transition table that prohibits re-entry into dispatch. In the crashed regime, the B3 ablation and AEP-full each have 540 executions: both record 0 undetected duplicates and 0 lost effects. Taking the run as the unit throughout, the individual one-sided 95% Wilson upper bound for either zero-event rate is 4.77% over 54 run clusters per arm. Declared ambiguity is 195/540 versus 193/540 (B3 minus AEP-full, 0.37 percentage points; stratified run-cluster 90% interval [-1.11, 2.04] percentage points). *Prevention* is what the barrier contributes, and the current evidence is narrower: in one NO-READBACK capability class, at one pre-acknowledgement Redis-kill point, on one host, its unwanted-applied-effect rate is 0.3333 versus 0.9333 over 30 runs per system ($10/30$ versus $28/30$, Fisher $p = 1.9 \times 10^{-6}$) — 18 real non-idempotent effects not committed, replicated over 4 further sessions at [6.1, 28.4] effects prevented.

The barrier's durability claim, which a process kill cannot exercise at all because `appendfsync everysec` defers the `fsync(2)` and not the `write(2)`, we test directly by making the block device stop accepting writes: across 3 independent replications, separation is perfect in every one — acknowledged records survive 90/90 and unacknowledged ones are destroyed 90/90 — against $0/10$ lost under a process kill on the same probe. Because detection does not depend on the barrier, its cost is a deployment choice rather than the protocol's price, and we give three measured points on that curve.

***Index Terms*—Fault tolerance, autonomous agents, idempotence, distributed coordination, write-ahead logging, fault injection, reliability engineering.**

Manuscript prepared September 4, 2026. Artifact: available via the submission system.

I. Introduction

An autonomous agent is asked to post a journal entry, issue a refund, or file a regulatory notification. It calls an enterprise API written years before agents existed. The call times out. The agent does not know whether the money moved.

This is an old problem in a new setting, and the new setting removes the assumption that the old solutions rest on. Durable-execution engines, workflow systems and message brokers achieve their guarantees by requiring that the operation being retried is idempotent, or that the remote side will accept a deduplication key, or that both endpoints can be enrolled in one transaction [1], [2], [3]. Temporal's own documentation is explicit about the precondition: "You should always make your business logic Activities idempotent in Temporal. Because Activities may be retried, these functions may be executed more than once" [4]. The legacy endpoints that agents are now being pointed at satisfy none of this. They are non-idempotent, they accept no idempotency key, and — the property that does the real damage — many of them cannot be asked afterwards whether a mutation was applied.

A. The trilemma

Consider a worker that has sent a mutation and then crashed, and a recovery process that must decide what to do. Exactly three things can happen to the external world, and a system must choose which one it is willing to produce.

- An ***undetected duplicate***. The recovery process re-sends. If the first request had in fact been applied, a second real effect now exists and no component of the system knows it. This is what a naive retry produces, and, as we show in Section VI, it is also what a lease, a fenced compare-and-swap, and a durable event-sourced history each fail to prevent, because none of them is a record written *before* the call.

- A ***lost effect***. The recovery process does not re-send and records the attempt as failed. If the first request *was* applied, a real effect now exists that the system's records deny. This is what an at-most-once configuration produces. It is quieter than a duplicate and it is not better; a refund that the ledger says did not happen is a reconciliation problem that surfaces weeks later, in the accounts.
- A ***declared ambiguity***. The system declines to decide, records that it cannot decide, and stops. The effect may or may not exist; what is guaranteed is that a durable record says so and that no further automated action is taken on that execution.

The first two are silent. The third is not. Our position is that the third corner is the one a protocol should be engineered to reach, and that the quantity a system should be judged on is not whether it avoids ambiguity — which, against an endpoint that cannot be queried, is impossible — but whether its residual uncertainty is *declared and bounded* rather than *silent and unbounded*.

The impossibility is not incidental. Deciding whether a remote effect occurred, when the channel may fail and the remote side may not answer, is the coordinated-attack problem; no protocol resolves it with certainty, and the classical results on consensus under crash faults and on failure detection say why [5], [6]. What is available is a choice about where the uncertainty is allowed to surface. AEP's answer is: in a durable state, visible to an operator, never in the accounts.

B. Contributions

C1 **The declared-ambiguity formulation.** We state the problem as the three-way trade of Section I-A rather than as a pursuit of exactly-once, and we give three properties — fenced state, detectable ambiguity, and a fail-closed liveness bound — each mapped to the code path that enforces it and each with its residual window declared rather than assumed away (Sections III and IV).

C2 **A protocol and an implementation** in which a durably acknowledged write-ahead intent is a *checked precondition* of dispatch authority rather than a matter of call ordering: under a trusted-code assumption, the fsync acknowledgement returns an opaque, non-copyable, single-use, scope-bound in-process dispatch guard; the guard is consumed to write a Redis-visible authorization, and the pre-dispatch script refuses to proceed without it (Section IV). This is a control-flow invariant of the supported API, not a cryptographic capability or a boundary against arbitrary code in the same Python process.

C3 **An evaluation under real process kills** across six named crash points, three endpoint reconciliation capabilities and three collected fault regimes — every execution crashed, none crashed, and Redis hard-killed — against five baseline designs, one of them in two configurations. (Two further regimes are implemented and were not collected; they are named in Section VIII rather than counted here.) AEP records no undetected duplicate and no lost effect in any cell measured; the baselines without a pre-dispatch record duplicate in most crashed executions. AEP's residual is declared ambiguity whose rate is a function of what the endpoint can be asked (Section VI).

C4 **A decomposition of the mechanism, by ablation, that reassigns our own headline result.** The write-ahead pattern is normally presented as one mechanism delivering one guarantee. It is two. The *pre-dispatch record plus no re-entry* is what makes an outcome detectable, and it does so without the durability barrier: ablating the barrier produces no observed difference in the crashed-regime detection metrics (Section VI-C1). The *barrier* buys something else — it withholds the external call when durability can no longer be confirmed — which is invisible to the crash faults that dominate this literature and large under the fault that targets Redis (Section VI-C2). Separating them changes what a deployment must pay for: detection is nearly free, prevention is where the fsync cost lives, and an operator can buy the first without the second (Section VI-D1).

C. What this paper does not claim

We claim no exactly-once external execution, no prevention of a duplicate the provider has already accepted, no high availability, consensus or split-brain immunity, and no durability beyond one local AOF fsync. The system is a single Redis trust domain and the measurements are single-node. These are stated as a table of non-claims in Section III with the reason each is out of scope, and revisited in Section VIII.

II. Motivating Traces

The four traces below are not hypotheticals. Each is a run of our harness (Section VI), replayed from its event log and cross-checked against the mock provider's ground-truth ledger — an independently written record of every mutation the provider *actually applied*. In each, the agent workload is the same: acquire work, call one non-idempotent endpoint, record the outcome.

A. Trace 1 — the duplicate nobody sees (B0, naive retry)

A worker sends the mutation. The provider applies it. The worker is `SIGKILL`ed after the response is on the wire but before the outcome is recorded (crash point `after_response_before_resolution`). A supervisor observes an execution with no terminal record and runs the step again. The provider applies the mutation a second time.

The ground-truth ledger now holds two applied mutations for one logical step. The system's own records hold one successful execution. Nothing in the system is in a state that a monitor could alert on: there is no error, no retry counter above its threshold, no ambiguous record. The execution is *green*.

This is not a strawman configuration; it is what a retry-on-timeout loop does, and it is the behaviour we measure for B0. Across the crashed cells B0 ends this way in 0.77–0.83 of executions depending on the endpoint, and so do B1 and B2 (Table VI).

B. Trace 2 — a lease and a fenced write do not help

The same crash, against B1 (a real Redis lease) and B2 (a lease plus an exact expected-version compare-and-swap on the state document). Both are the patterns an experienced engineer reaches for, and both are doing their job: no stale writer corrupts the state document, and no two workers hold the lock at once.

Neither prevents the second effect. The lease governs *who may write state*; the CAS governs *which version of state may be replaced*. The external call is neither. When the supervisor re-runs the step it holds a valid, freshly acquired lease and writes a correctly versioned record — of a duplicate. Section VI shows B1 and B2 duplicating at substantially the same rate as B0, which is the point: the mechanisms that protect the database do not protect the world.

C. Trace 3 — a durable log is necessary and not sufficient (B4)

B4 is an event-sourced durable-execution engine of the kind these systems are built on: an append-only history in Redis, each append acknowledged durable with the *same* `WAITAOF` barrier AEP uses, replay after a crash, and completed activities memoised so they are not re-run. It is not ablated on durability. Its history is on disk before the call.

It duplicates anyway. The history records that the activity was *scheduled*; it does not, and cannot, record whether the provider applied the effect, because that fact was never available to the engine. On replay the engine sees a scheduled activity with no completion and — following the vendor's documented default, where Activity Maximum Attempts is unlimited [7] — schedules it again.

The mitigation the vendor recommends is to make the activity idempotent [4]. That is exactly the precondition this paper's premise removes.

D. Trace 3b — and the at-most-once configuration loses the effect

Setting Maximum Attempts to 1 is documented and standard: "Setting the value to 1 means a single execution attempt and no retries" [7]. We run it as B4b. It cannot produce a caller-caused duplicate, and it does not.

What it produces instead is the other silent failure. Where the provider applied the mutation before the worker died, B4b's history records an activity that timed out; the workflow fails; the effect exists and the records deny it. Critically, that record is an ordinary failure and not a declared ambiguity — there is no outcome class in B4b in which the engine can say *I do not know*, so nothing escalates and no operator is told that a real-world effect may be unaccounted for.

E. What the traces establish

Table I is the shape of the argument. At-most-once dispatch is available off the shelf — B4b buys it with one configuration setting, and so does AEP's ablation B3. What is not available off the shelf is knowing *which* of duplicate-or-loss occurred, because the fact a system would need for that — whether the provider applied the effect — is not in any log it can write and cannot be put there by any amount of logging. It has to be obtained from the endpoint, and Section VI-B shows the price of that: the achievable outcome is bounded by what the endpoint can be asked.

TABLE I
THE TRILEMMA, AS THE SYSTEMS UNDER COMPARISON INSTANTIATE IT. A DURABLE WRITE-AHEAD RECORD IS NECESSARY (TRACES 1, 2) AND NOT SUFFICIENT (TRACE 3); WHAT DISTINGUISHES AEP IS AN OUTCOME CLASS IN WHICH IT CAN DECLINE TO DECIDE. B3 — AEP WITH THE DURABILITY BARRIER REMOVED AND NOTHING ELSE — REACHES THE SAME CORNER, WHICH IS THE POINT: THE BARRIER IS NOT WHAT BUYS DECLARED AMBIGUITY (SECTION VI-C1 MEASURES THE ABLATION), AND SECTION VI-C2 ISOLATES WHAT IT DOES BUY.

	undetected duplicate	*lost effect*	*declared ambiguity*
B0 naive retry	**yes**	—	no
B1 lease-only	**yes**	—	no
B2 CAS-only	**yes**	—	no
B4 durable, max attempts ∞	**yes**	—	no
B4b durable, max attempts $= 1$	no	**yes**	no
B3 AEP minus the barrier	no	no	**yes, bounded**
AEP-full	no	no	**yes, bounded**

III. SYSTEM AND FAILURE MODEL

This section is a condensation of `docs/22-formal-model.md` in the artifact, which states every claim below with a `file:line` citation to the enforcing code and is machine-checked in CI: a script fails the build if a cited path stops existing or a cited line falls outside its file. That check proves citation *ranges* are valid, not that the semantics are right; the semantic evidence is the per-property regression tests named in Section IV.

A. Principals and store

A *runner* performs at most one external mutation per invocation. A *recovery service* is read-only with respect to the external system and writes only coordination state. A *connector* is the only component that touches the external API. The store is a single self-hosted Redis 7.2 instance with the append-only file enabled and RDB snapshots disabled — no replica, no Sentinel, no Cluster.

Redis executes a Lua script without interleaving, so a script is a single serialisation point. This is the only atomicity primitive the protocol has and every invariant below is expressed inside one.

B. Clocks

There is no synchrony assumption beyond lease TTLs. All ordering decisions between principals are taken against the Redis server clock; no cross-worker clock comparison occurs. Process-local monotonic time is used only for heartbeat intervals and watchdogs, never for an authoritative decision. The lease budget $T_{\text{client}} \leq T_{\text{lock}} - B$ with $B \geq 15\,\text{s}$ is enforced

TABLE II
ENDPOINT RECONCILIATION CAPABILITY. THE RIGHT-HAND COLUMN IS THE CEILING ON WHAT *any* PROTOCOL CAN CONCLUDE, AND SECTION VI-B SHOWS AEP'S DECLARED-AMBIGUITY RATE TRACKING IT.

Capability	Conclusions the endpoint can support
AUTHORITATIVE READBACK	applied, not-applied, unknown, conflict. Absence *proves* the mutation did not occur.
POSITIVE-ONLY READBACK	applied, unknown, conflict. Presence can be confirmed; absence proves nothing.
NO-READBACK	none. The endpoint cannot be queried at all.

TABLE III
THE SIX CRASH POINTS, AND WHAT EXISTS IN THE WORLD AT EACH. THE RIGHT-HAND COLUMN IS WHY THE DECLARED-AMBIGUITY RATE IN SECTION VI-B VARIES SO SHARPLY ACROSS THEM.

Crash point	State when the worker dies
`before_intent_write`	no record, no effect
`after_intent_before_barrier`	record in memory, maybe not on disk; no effect
`after_barrier_before_dispatch`	record durable; **no effect**
`mid_dispatch`	record durable; effect **likely**
`after_response_before_resolution`	record durable; effect known to the dead process only
`after_resolution_before_barrier`	outcome written, not yet acknowledged

TABLE IV
NON-CLAIMS, WITH THE REASON EACH IS OUT OF SCOPE.

Non-claim	Why
Exactly-once external execution	Redis and a legacy provider share no transaction coordinator. The protocol converts this into declared ambiguity.
Duplicate *prevention*	AEP prevents *automatic* duplicates. An effect the provider already accepted cannot be recalled.
HA, consensus, split-brain immunity	Single instance; the lock is a lease [9]. This is also why the AOF-rewind residual below has no local fix.
A hardened trust domain	No Redis ACL; any principal reaching the socket can write the state key directly.
Durability beyond one local AOF fsync	The barrier requires one *local* fsync [10]; it is explicitly not a substitute for replication.
Operator alerting	Not implemented. "Escalates" means *reaches a terminal automated state and pauses the execution*, nothing more.
Machine-checked proofs	Properties are argued from code paths, not model-checked.

at configuration time, and it is a timing budget rather than a guarantee: a scheduling gap remains between a successful preflight and transmission, and we declare it as such.

C. The external endpoint

The endpoint is non-idempotent and non-cooperative: it need not accept an idempotency key, and its response may be absent, late, or self-contradictory. Two things about it are modelled explicitly, and the second is the axis the evaluation turns on.

a) Evidence at mutation time: The runner accepts only policy-declared success and failure values, required non-empty and disjoint. Any other value, and *any* exception, is coerced to ambiguity. Ambiguity is the default, not a special case.

b) Reconciliation capability: What the endpoint can be asked *afterwards* is a declared, typed property with three values (Table II). The declaration fails closed: a missing, null or unrecognised capability is treated as the weakest, not as the strongest. Classification is total — in particular, "the endpoint reports the mutation absent" combined with a positive-only capability is a *named contract violation* that resolves to ambiguity, not a fall-through to "it did not happen".

D. Failure model

Crash-and-delay, not Byzantine. Redis is trusted to execute scripts atomically and honour expiry; the provider is trusted only insofar as its declared evidence is truthful.

- **F1 Worker crash at any instruction boundary.** Six named crash points around the external call (Table III). In the evaluation these are OS-level `SIGKILL` of a separate worker process, not in-process exceptions.
- **F2 Partition between a worker and Redis.** Any Redis command may raise; every raise forbids progress rather than permitting a guess. A partition outlasting the lease is observationally equal to F5.
- **F3 Redis restart with AOF replay.** Up to roughly one second of acknowledged-but-unsynced writes may be absent under `appendfsync everysec` [8]. Section VI-C3 reports two probes that sharpen this considerably: the set of *faults* that can actually realise that loss is narrower than the documentation's wording suggests — a process kill is not in it — and a host-level write loss, which is, realises it every time.
- **F4 Delayed or duplicated external responses.** A late effect cannot be recalled; recovery waits out a reconciliation delay before drawing any conclusion, and never re-dispatches.
- **F5 Worker pause past lease expiry** (GC or VM stall), handled by an atomic pre-dispatch preflight, cancellation of the protected task on ownership loss, and CAS rejection of the late write.

Out of model: Byzantine Redis; a provider whose declared evidence contradicts its own effects; loss of the Redis host *after* fsync acknowledgement; adversarial writers with direct socket access; compromise of vault keys.

E. Non-claims

We state these because reviewers are right to look for them, and because several are structural rather than incidental.

IV. The AEP Protocol

A. Ordering

The runner's sequence, with the property each step exists to serve:

1) Acquire the lease with jittered retry.
2) Require that execution state already exists.
3) Prepare, vault and read back the request *before any intent exists*, so that the intent, once written, names a request that is already durable.
4) Write the intent NONE→ABOUT-TO-FIRE by fenced CAS, **and** run the durability barrier on the *same pinned connection*.
5) Convert the barrier's acknowledgement into a Redis-visible dispatch authorization.
6) Run the atomic pre-dispatch preflight, which re-checks that authorization.
7) Mint a single-use dispatch capability.
8) Only then call the connector.

If the barrier fails, the runner makes one fenced attempt to record FAILED-CONFIRMED with evidence `LOCAL_NO_DISPATCH`, falling back to leaving the intent for recovery. No bytes were sent, so a confirmed failure is sound there — and this is the path that produces the measured result of Section VI-C2.

B. P1 — Fenced state

Property 1 (Fenced state): No committed state write can be superseded and then resurrected by a stale writer. Every write to the state key is admitted only if, in one atomic Lua invocation, (a) the caller's ownership token equals the live value of the lock key, and (b) the stored version equals the caller's expected version and the candidate's version equals expected $+ 1$.

Both conjuncts are necessary and the failure of either is a known pattern. Version-only CAS admits a writer that lost its lease but holds the current version; token-only checking admits a writer rebasing on a stale read. The pairing is the fencing-token argument made against lease-only locking [11], with the token check and the write placed inside one script so they cannot interleave. The same script additionally freezes, inside the atomic region, the execution identity, the schema version, every envelope field outside a declared mutable set, every other ledger entry, and the intent's immutable fields; transitions are append-only with a prefix check, and ledger entries can never be deleted.

a) Declared residuals: A plain `SET` from any client with socket access is unconstrained (there is no ACL). Expiry, not deletion, ends the guarantee: after the retention floor elapses the record is gone and any writer may create version 1 again. And — confirmed by a probe rather than argued — an AOF rewind can un-fence: P1 is a statement about one monotonic Redis timeline, and because lock keys are deliberately *not* barriered, a lease whose release was lost can reappear alongside a rewound version, so a previously fenced writer satisfies both conjuncts. There is no local fix; the fix is consensus, which is a non-claim (Table IV).

C. P2 — Detectable ambiguity

Property 2 (Detectable ambiguity): If a *durable* ABOUT-TO-FIRE intent exists, then under F1–F5 the intent is eventually assigned exactly one of FIRED-CONFIRMED, FAILED-CONFIRMED or PERMANENTLY-AMBIGUOUS — never silently re-dispatched and never silently dropped — provided the recovery loop runs, survives its own pass, and can eventually obtain the lease.

The design question P2 raises is what makes "barrier before dispatch" more than a convention. In this protocol it is a checked precondition, in four links:

1) Through the supported API, the barrier issues a `DurabilityAck` only when `WAITAOF` returned at least one local fsync.
2) Ordinary callers cannot construct, subclass or copy `DurabilityAck`; the in-process registry detects reuse and binds the guard to `execution:intent:prepared_version`, so a guard for one attempt cannot authorise another through the supported path.
3) Authorizing dispatch *consumes* the ack, then runs a script that re-checks lease, version, status and binding digest and writes an authorization key.
4) The preflight script requires that exact value; an absent authorization defaults to a value that cannot match, so it fails closed.

a) The irreducible gap, stated: Redis exposes no way for a Lua script to verify that a previous `WAITAOF` succeeded. A complete in-store proof is therefore impossible. Under trusted-code assumptions, an authorization records that the supported path received and consumed its ordinary barrier guard; it is not an independent proof that Redis fsynced. The HMAC and registry are mutation/reuse checks, not a cryptographic security boundary. Python underscore naming and closure placement do not prevent arbitrary same-process code from calling the module-internal issuer without running the barrier. A principal with direct Redis access can also write the authorization key, and a caller inside the process can compose the binding service with a connector directly, bypassing the runner. A stronger attacker model requires a separately enforced process or service boundary, which this implementation does not have.

b) Recovery classification: For an eligible intent, under a freshly acquired lease and after re-validating status and version, the mapping from observation to outcome is total (Table V). Recovery has no reference to the mutation call at all — it can only read back — and the transition table has no edge back to ABOUT-TO-FIRE, checked in Python and re-checked inside the atomic script.

c) False-positive ambiguity is the price, and it is the right direction: A crash after a durable intent but *before* transmission yields an intent that recovery must treat as ambiguous even though no effect exists. This inflates the declared-ambiguity rate; it can never inflate the undetected-duplicate rate. Section VI-B shows this asymmetry directly in the data: ambiguity is highest at exactly the crash point where no effect can possibly exist.

TABLE V
RECOVERY CLASSIFICATION. THE TWO ROWS IN BOLD ARE WHERE DECLARED AMBIGUITY IS PRODUCED BY DESIGN RATHER THAN BY FAILURE, AND THEY ARE WHAT SECTION VI-B MEASURES.

Observation	Outcome
capability undeclared / unrecognised	**ambiguous**, no query
NO-READBACK	**ambiguous**, no query
applied (either querying class)	confirmed
not-applied + AUTHORITATIVE	refuted
not-applied + POSITIVE-ONLY	**ambiguous** (named violation)
conflict	**ambiguous**
unknown, budget remaining	stay unresolved, backoff
unknown, budget exhausted	**ambiguous**

D. P3 — Fail-closed liveness bound

Property 3 (Fail-closed bound): Automated reconciliation of one intent terminates within a configurable attempt budget and duration budget, after which the intent is placed in a terminal automated state and the execution is withdrawn from normal scheduling.

Defaults are 8 attempts and 24 hours, with full-jitter backoff. Attempt counting is monotone and re-checked inside the atomic script: an unresolved-to-unresolved edge must increment the counter by exactly one. On exhaustion the intent becomes PERMANENTLY-AMBIGUOUS, the execution is paused, no new intent may be created for it, and the only exit is an operator transition that itself requires the lease and the exact version.

a) Declared residuals: The bound is consulted only on the *unknown* branch — every other observation resolves immediately, so this is not a gap in practice, but it is not a global watchdog either. Lease contention does not consume budget. There is no escalation *mechanism*: "ejects to operator escalation" means reaching a terminal state and pausing, and nothing alerts. And escalated records still expire: a retention floor of 31 days is enforced for any ledger containing an unresolved or ambiguous record, but a floor is not permanence, so after it elapses the escalation evidence is deleted without an operator having acted. Both residuals are confirmed by executable probes in the artifact, not asserted.

E. The state machine

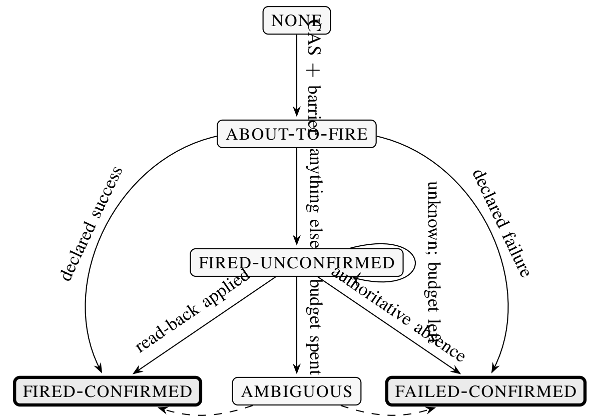

Fig. 1. The intent state machine, generated from the transition table in `aep_core/core/intents.py`. Solid edges are automated; the dashed edges from AMBIGUOUS are the operator-only transitions. There is no edge back into ABOUT-TO-FIRE — that absence is what "never silently re-dispatched" means, and it is enforced inside the atomic script rather than by the caller.

V. IMPLEMENTATION

AEP is 6 862 lines of Python 3.13 against Redis 7.2.5, pinned by image digest. The evaluation harness, mock provider and six baseline systems are a further 23 803 lines.

a) What the protocol rests on: Three Lua scripts carry the intent path, each a single serialisation point: the intent compare-and-swap that holds the whole transition table, the pre-dispatch preflight, and the dispatch-authorization writer. Three further scripts implement the lock's release and renewal and the generic document compare-and-swap; they are not on the dispatch path and none of the properties of Section IV rests on them. A Lua re-implementation of the strict UTF-8 and duplicate-JSON-member checks is prefixed to every authoritative script, so the store rejects a malformed document rather than trusting the client that sent it.

b) Composition modes: A start-up gate distinguishes TEST, EVALUATION and PRODUCTION compositions and refuses to run a mismatched one. PRODUCTION and EVALUATION share one block of requirements and differ only in the connector endpoint, which makes "the measurements ran in a production-shaped composition" a property of the code rather than of this sentence. PRODUCTION is nevertheless *unsatisfiable* in this repository — there is no production vault and no production connector, and the gate fails closed for it. Every number in Section VI was therefore collected in EVALUATION mode, with no test-only flags anywhere in the harness, and the evaluation vault's two weaknesses relative to the production design (it shares the Redis trust domain, and its keys are operator-supplied rather than KMS-wrapped) are declared rather than hidden.

c) On test counts: The artifact carries a large automated suite and CI that fails if any test is skipped. We deliberately do *not* offer the test count as evidence of protocol assurance. A prior internal audit of this repository established that the suite had grown chiefly in rejection branches, gate scripts and artifact metadata, while the number of tests exercising the write-ahead path was close to unchanged; a reader who reads a doubled test count as a doubled protocol guarantee would be wrong. The evidence for the protocol's behaviour under faults is Section VI, and the harness found three defects the suite had missed — discussed in Section VIII because two of them would have biased results in AEP's favour had they gone unnoticed.

d) Reproducibility: The environment is locked (`uv.lock`), the Redis image is pinned by digest, and CI provisions Redis from the same compose file the experiments use, so no job can pass against a differently configured store. Continuous integration runs four jobs — the full suite, the `WAITAOF` durability suite, a citation-range check over the formal model, and a gate that rebuilds the manuscript and re-derives every number in it from the frozen results — and the build fails if any test is skipped.

VI. Evaluation

RQ1 Under crashes, does AEP eliminate *undetected* duplicates, and what does its residual ambiguity depend on?

RQ2 Which of the protocol's two durability mechanisms produces which guarantee — and against which faults is each one observable?

RQ3 What does the protocol cost, and how much of that cost is a choice?

RQ4 How does recovery behave?

A. Setup

a) Systems: The seven of Table I, implemented as thin variants over one connector, one workload driver and one oracle, so that only the differences differ. Each carries a machine-readable descriptor that tests check against the implementation, and a crash point that does not exist in a given system is marked inapplicable and never run rather than silently skipped.

b) Faults: Workers are separate OS processes killed with real `SIGKILL` at one of the six crash points of Table III. A *regime* is a named fault condition — not a matrix dimension — and we report each separately because they are different experiments:

- **crashed**: every execution is killed at the cell's crash point. This is the regime RQ1 is about.
- **crash-free** (`p0`): nothing is injected. The only regime RQ3 may use.
- **`redis-kill-preack`**: no worker crash; Redis is hard-killed with `docker kill -s KILL` at the instruction boundary between the intent CAS and the barrier acknowledgement, then restarted and verified. This is RQ2's ablation.

c) Oracle: A standalone mock provider records every *actually applied* mutation in a SQLite ledger (WAL, `synchronous=FULL`, one transaction per mutation). Duplicates are counted from that ledger, never from the system's own opinion of itself. The analysis is permitted to read exactly two files per run — the event log and the oracle ledger — and a source gate that parses the analysis modules fails the build if they import anything that could reach live protocol state.

d) Statistics: Rates are over executions. Per-cell rate intervals use a seeded run-cluster bootstrap (10 000 resamples); the B3–AEP ambiguity difference uses a stratified run-cluster bootstrap that preserves crash-point, endpoint-capability, and keying strata. Fisher's exact tests compare execution counts and therefore do not adjust for run clustering; we report them descriptively, not as evidence of equivalence. Seeds and repetition counts are recorded per run.

e) Two reporting rules we hold ourselves to: First, *no pooled table*. A rate computed across fault regimes is a property of how many runs of each kind were collected, not of any system; our own analysis tool prints a warning whenever more than one regime is present, and every number below names its regime. Second, *no absolute timing from an ungated host*. A run contributes a duration only if the operator declared before the run that the host cannot suspend *and* the run's wall-versus-monotonic divergence stays within tolerance. Both halves are necessary: a host that merely was not idle long enough to suspend is indistinguishable from one that cannot. Under this gate, *zero* runs collected before this rule existed contribute a timing number, and none does. Both rules were adopted after the first collection — the pooling rule when a mid-study review found the pooled summary table unusable as a source, the timing gate after a host suspension was discovered mid-run — and every rate and duration below is reported under them.

B. RQ1 — undetected duplicates and the shape of the residual

Table VI is the anchor result; Figure 2 shows the same executions pooled to one bar pair per system.

Three readings, in the order they matter.

a) AEP's two silent columns are empty, everywhere measured: Across 180, 180 and 180 crashed executions on the three capability classes, AEP-full recorded an undetected-duplicate rate of 0.0000, 0.0000 and 0.0000, and a lost-effect rate of 0.0000, 0.0000 and 0.0000. Every one of those executions was killed by `SIGKILL` at a named point around the external call. This was the pre-registered halt condition of the experiment: a single undetected duplicate in AEP-full would have stopped the matrix and become the paper's primary result. It did not occur.

b) The baselines without a pre-dispatch record duplicate in most crashed executions: B0, B1 and B2 sit between 0.77 and 0.83, and the weakest of the three comparisons against AEP-full is significant at $p = 5.4 \times 10^{-182}$ by Fisher's exact test. The gap between B1/B2 and B0 is small and that is the finding: a lease and a fenced CAS are doing exactly what they promise and neither promise is about the external call (Figure 3).

c) A durable log is not enough, and the two configurations of one engine land on two different corners: B4 has a write-ahead record, acknowledged durable by the same `WAITAOF` barrier AEP uses. It is not ablated on durability. It duplicates at 0.5889 on POS-ONLY and 0.5611 on NONE (180 and 180 executions), and it duplicates on AUTH as well, at 0.5278 over 180 executions. The endpoint's reconciliation capability does not rescue it, which is the point — a read-back tells a recovering workflow what the provider currently holds, and B4's duplicate is a second application it chose to make, not a fact it failed to look up.

B4b is the same engine with Maximum Attempts set to 1, and it is the cleaner half of the demonstration: over 180 and 180 executions it records *zero* undetected duplicates on both classes and loses the effect in 0.5056 and 0.5167 of them instead, and 0.5444 over 180 on AUTH. One configuration line moves the engine from one silent corner of Section I-A to the other, and neither configuration reaches the third: across all 36 B4 and B4b cells the declared-ambiguity rate never exceeds 0.0000.

That is the sharpest form of the argument the paper can make, because it is made against a system that has everything the write-ahead pattern is supposed to provide. The record is necessary and it is not sufficient, because the record cannot

TABLE VI

The trilemma, measured. Every execution in every cell was killed at one of the six crash points of Table III; rates are over executions, pooled across crash points within one endpoint capability. B0–B2 pool over five of those six rather than all of them — `after_intent_before_barrier` cannot occur in a system that writes no intent, so for those three there is no such cell to pool — and the per-crash-point rates behind every cell here are in `per-cell-metrics.csv`. AUTH/POS-ONLY/NONE are the reconciliation capabilities of Table II. AEP-full and B3 — the same protocol with and without the durability barrier — are the only systems that record any declared ambiguity, and the only two whose undetected-duplicate and lost-effect columns are zero throughout. Their equality in every cell shown is this paper's ablation result rather than an accident of this table, but the evidence for it is a bound obtained by pooling these columns, not by testing the cells individually: Section VI-C1 puts both zero-event rates in $[0, 4.77\%]$ at joint coverage of at least 90%. The per-class cells this table is organised by are not separately bounded — at that scope the width would be 20.1 percentage points — so what the table shows per class is no observed difference, and Section VI-C1 is where the claim is quantified. Source: `per-cell-metrics.csv`, crashed regime only.

	undetected duplicate			lost effect			declared ambiguity		
system	AUTH	POS-ONLY	NONE	AUTH	POS-ONLY	NONE	AUTH	POS-ONLY	NONE
B0 naive retry	0.8200	0.8000	0.7733	0.0133	0.0133	0.0067	0.0000	0.0000	0.0000
B1 lease-only	0.8133	0.8067	0.8267	0.0000	0.0067	0.0067	0.0000	0.0000	0.0000
B2 CAS-only	0.8067	0.7933	0.7800	0.0067	0.0067	0.0000	0.0000	0.0000	0.0000
B4 durable, ∞ attempts	0.5278	0.5889	0.5611	0.0111	0.0000	0.0111	0.0000	0.0000	0.0000
B4b durable, 1 attempt	0.0000	0.0000	0.0000	0.5444	0.5056	0.5167	0.0000	0.0000	0.0000
B3 intent, no barrier	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3667	0.7167
AEP-full	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3500	0.7222

TABLE VII

AEP-full's declared-ambiguity rate, by crash point and endpoint capability. The rate is not a constant of the protocol: it is zero where the crash precedes the intent write (nothing was promised), zero throughout AUTH (absence is provable), and highest exactly where no effect can exist but the endpoint cannot say so. Undetected duplicates and lost effects are 0 in every cell of this table.

crash point	AUTH	POS-ONLY	NONE
`before_intent_write`	0/30	0/30	0/30
`after_intent_before_barrier`	0/30	30/30	30/30
`after_barrier_before_dispatch`	0/30	30/30	30/30
`mid_dispatch`	0/30	0/30	30/30
`after_response_before_resolution`	0/30	1/30	30/30
`after_resolution_before_barrier`	0/30	2/30	10/30

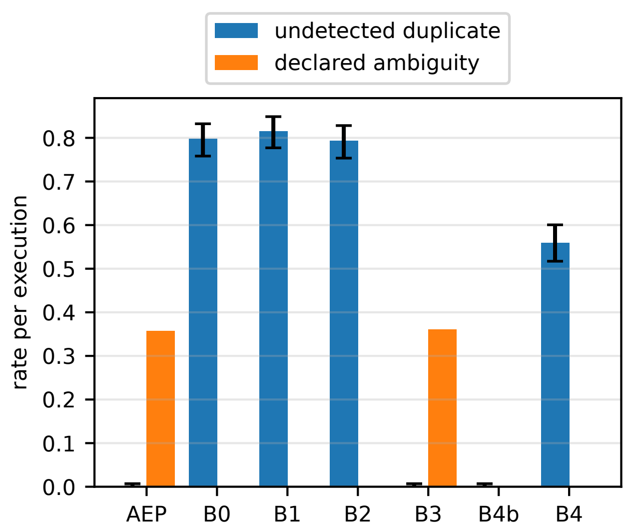

Fig. 2. The trade, per system. Crashed regime only, pooled over crash points and endpoint capabilities weighted by executions. The three systems on the right have an empty left bar; only AEP-full and its ablation B3 have a right bar at all. Interval is Wilson on the pooled counts — the cluster-aware bootstrap intervals, which are wider, are in `per-cell-metrics.csv` and are the ones to quote.

contain the fact that matters — whether the provider applied the effect. What separates B3 and AEP-full from B4 is not durability but the *absence of an edge back into the dispatching state*: the transition table has no path that re-enters ABOUT-TO-FIRE, checked inside the atomic script (Figure 1). What separates them from B4b is that declining to re-send is not the same as knowing whether to.

1) The residual is set by the endpoint, not the protocol: AEP's declared-ambiguity column is 0.0000 on AUTH, 0.3500 on POS-ONLY and 0.7222 on NONE. That ordering is not a defect that better engineering would remove; it is Table II showing through. Where the endpoint can prove absence, recovery resolves everything and nothing is left ambiguous. Where it can only confirm presence, absence is uninformative and the honest answer is "I do not know". Where it cannot be queried at all, no amount of protocol recovers the fact.

Table VII decomposes this further, and it is where the protocol's conservatism becomes visible.

Two cells carry the argument. At `before_intent_write` the rate is zero in every capability class: the crash precedes the intent, nothing was promised, and there is nothing to be ambiguous about. At `after_barrier_before_dispatch` it is at its maximum — and that is the crash point at which *no effect can possibly exist*, because the worker died before sending anything. AEP declares ambiguity most often exactly where it is least warranted by the world. That is the false-positive ambiguity of Section IV-C, and its direction is the whole design: the protocol's errors are toward saying "I might have done something" when it did not, never toward silence when it did. The cells where an effect *does* typically exist — `mid_dispatch` and `after_response_before_resolution` under POS-ONLY — show low ambiguity, because a positive read-back confirms the effect and the intent resolves.

2) Why the provably-empty cell is not resolved, and what it would cost: The `after_barrier_before_dispatch` row invites an obvious question: if the worker died before sending anything, *the system could know that*. Why is the outcome ambiguous rather than a confirmed non-event?

Because nothing durable records it. The last durable write before the call is the dispatch authorization, and the runner then performs preflight, mints the capability and calls the connector without touching Redis again. A recovery process reading the store afterwards sees an authorized, unresolved intent — and that is exactly what it would see had the worker died *during* transmission. The two histories are identical in the store, so recovery fails closed, which is the correct behaviour given what it can observe.

Closing the gap is possible and we did not do it: a third durable barrier immediately before transmission would split the two cases and let recovery refute the earlier one. The measured price of a barrier on this configuration is 983.3 ms (Section VI-D), so that is a 24.6% increase in step latency to convert one crash point's declared ambiguities into confirmed non-events — and it narrows rather than closes the window, since a crash between that barrier and the socket write returns the same ambiguity. We record the trade rather than take it, and note that its value depends entirely on the deployment's ratio of crash points, which our uniform matrix deliberately does not model.

3) The cost of the third corner: Declared ambiguity is not free, and a paper that presented it as a pure win would be selling something. Each declared ambiguity is an execution that has stopped, that will not proceed without a human, and whose resolution requires someone to establish out-of-band what the endpoint will not say. On a NO-READBACK endpoint under a crash-heavy regime that is most of the crashed executions. What we claim is not that this is cheap but that it is *the honest price of the information the endpoint withholds*, and that the alternative is not fewer incidents but the same incidents discovered later by a different department. We have not evaluated whether operators handle declared ambiguities well, or how much a real deployment would generate; both are stated as gaps in Section VIII.

C. RQ2 — which mechanism produces which guarantee

AEP contains two durability mechanisms and the literature usually treats them as one. The first is the *pre-dispatch record*: an intent, written and fenced, that exists before the external call does. The second is the *barrier*: the `WAITAOF` round trip that makes the record's durability a checked precondition of dispatch authority (Section IV-C). B3 is the ablation that separates them — AEP-full with the barrier removed and nothing else removed — and running it against three different fault classes assigns each guarantee to the mechanism that actually produces it.

TABLE VIII
THE BARRIER ABLATION ON THE DETECTION METRICS. B3 IS AEP-FULL WITH THE WAITAOF BARRIER REMOVED AND NOTHING ELSE REMOVED. CRASHED REGIME; RATES ARE OVER EXECUTIONS, POOLED ACROSS CRASH POINTS WITHIN ONE CAPABILITY CLASS. THE P-VALUES ARE EXECUTION-LEVEL FISHER TESTS OVER ALL CAPABILITY CLASSES; THEY DO NOT ACCOUNT FOR RUN CLUSTERING AND ARE NOT USED AS EQUIVALENCE EVIDENCE.

metric	capability	AEP-full	B3	p
undetected duplicate	AUTH	0.0000	0.0000	1.00
	POS-ONLY	0.0000	0.0000	
	NONE	0.0000	0.0000	
lost effect	AUTH	0.0000	0.0000	1.00
	POS-ONLY	0.0000	0.0000	
	NONE	0.0000	0.0000	
declared ambiguity	AUTH	0.0000	0.0000	0.95
	POS-ONLY	0.3500	0.3667	
	NONE	0.7222	0.7167	

The assignment is not the one we expected, and it is the paper's central structural result.

1) Detection is the record's, not the barrier's: Everything in Section VI-B — zero undetected duplicates, zero lost effects, a residual of declared ambiguity shaped by endpoint capability — survives the barrier's removal intact.

Over 540 crashed executions per arm, B3 and AEP-full show no observed difference in either silent-failure metric. Both record an undetected-duplicate count of 0 and a lost-effect count of 0. Declared ambiguity is 195/540 for B3 and 193/540 for AEP-full; per capability class the two systems track each other: 0.0000 against 0.0000 on AUTH, 0.3667 against 0.3500 on POS-ONLY, 0.7167 against 0.7222 on NONE.

For ambiguity, B3 minus AEP-full is 0.37 percentage points. The stratified run-cluster 90% interval is $[-1.11, 2.04]$ percentage points, which lies within a ± 5 percentage-point margin. We introduce that margin in this revision; it was not preregistered, and we state it as a stipulation rather than derive it. We do not claim it is immaterial to an operator: Section VIII records that we have not evaluated declared ambiguity as an operational outcome, and that the artifact has no escalation mechanism, so we have no basis on which to price one. The result supports equivalence under that stated, post hoc margin, but only for this fixed design. Each arm has 54 run clusters distributed over 18 matched strata—three runs per stratum—so the cluster-aware interval is a sensitivity analysis, not a strong general equivalence result. The execution-level Fisher value ($p = 0.95$) is not used as equivalence evidence.

A word on what the zeros can and cannot support. Fisher's exact test on $0/540$ against $0/540$ returns $p = 1.00$ for duplicates and $p = 1.00$ for lost effects, and both numbers are worth nothing: a test between two zero counts has no power, and failing to distinguish them is not evidence that they are the same. What two zeros do support is a *bound*. The unit for that bound is the run, not the execution: each arm's 540 executions are 54 runs of ten, and ten executions that share a run, a crash point and a system are not ten independent trials. The one-sided 95% Wilson upper confidence bound on a zero numerator over 54 run clusters is 4.77%. This is a genuinely

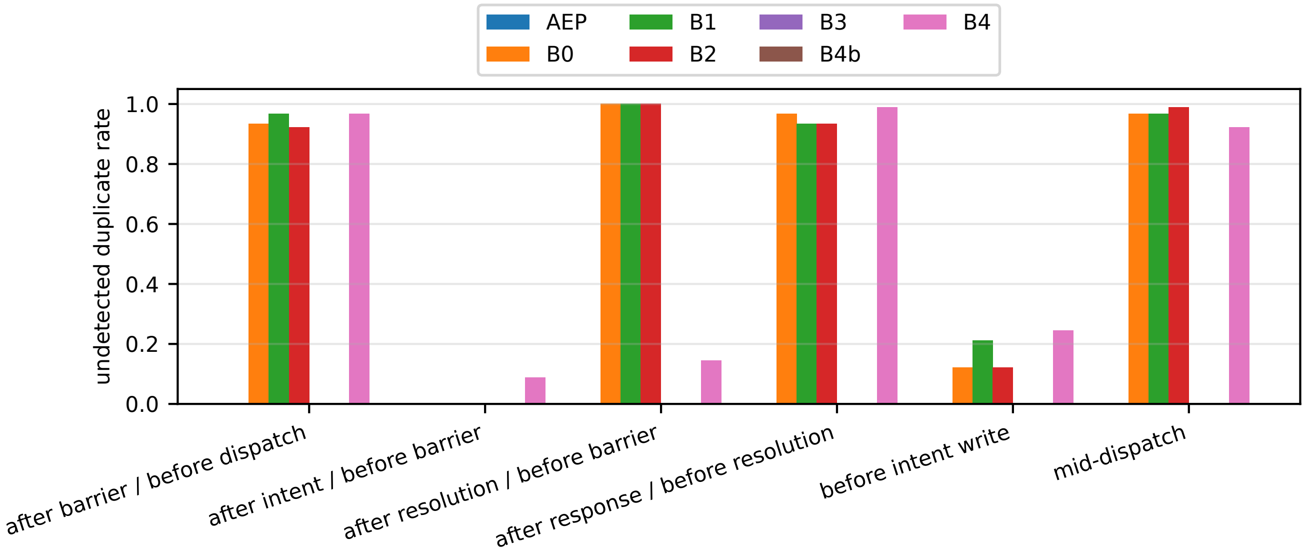

Fig. 3. Undetected duplicate rate by crash point, crashed regime, pooled over endpoint capabilities weighted by executions. The baselines' rate is not concentrated at one crash point: any interruption after the request is on the wire is enough, which is why a mechanism that acts only *after* the call cannot help.

one-sided bound using the 95th normal quantile; it is not the upper endpoint of a two-sided 95% Wilson interval, which uses the 97.5th quantile. Each system's rate is individually bounded this way. When the two system bounds are stated simultaneously, Bonferroni gives joint coverage of at least 90%, not 95%. Under that simultaneous event, both rates lie in $[0, 4.77\%]$, so their absolute difference is at most 4.77 percentage points. This finite-sample bound is the support for "no observed difference" in the two zero-event metrics; the corresponding Fisher values are not evidence of equivalence.

Treating the executions as independent would instead give 0.50%, and we do not report that as the bound. It is the same execution-level independence we decline for the Fisher values in Table VIII, and because no events were observed there is nothing in these data from which to estimate the within-run correlation that would license it. We state plainly that the choice of unit decides nothing we go on to claim: the baseline duplicate rates this bound is contrasted against are 77–83%, more than an order of magnitude from either figure, so the conservative unit is taken because it is the defensible one and not because the comparison needs it.

We report this as a finding rather than as a limitation, because it reassigns our own headline claim. The property that distinguishes AEP from B0, B1 and B2 — and the property the paper is named for — is produced by the pre-dispatch record and by the transition table that refuses to re-enter ABOUT-TO-FIRE. It is not produced by waiting for `fsync`. A reader who wants the detection guarantee and not the barrier's cost can have it, and Section VI-D1 prices that configuration explicitly.

This also sharpens what B4 shows. B4 has a durable record, acknowledged by the same barrier, and duplicates at the highest rate in the study; B3 has a durable record, no barrier, and duplicates at zero. Durability is neither necessary nor sufficient for detection. What is necessary is that the record be written *before* the call and that no path re-enter the dispatching state.

2) Prevention is the barrier's, and it needs the right fault: If detection does not need the barrier, what does the barrier do? It withholds the external call when the durability of the intent can no longer be confirmed. That is a different guarantee, it is worth stating in its own metric, and no worker-crash fault can exercise it — which is why Section VI-C1 returns a null. The fault that exercises it is loss of Redis itself.

The metric is the *unwanted-applied-effect rate*: the fraction of executions in which a real, non-idempotent mutation reached the provider while the durability of its own intent record could no longer be established. It is not the same quantity as an undetected duplicate and it is not measured on the same regime; keeping the two apart is the whole point of this subsection.

We hard-kill Redis at `after_intent_before_barrier`: the instruction boundary between the intent CAS and the acknowledgement. AEP-full is inside `WAITAOF` at that moment; B3 is not, because B3's ablation is precisely that round trip. One execution per run, no worker crash — the fault under study is Redis dying.

a) The controlled fault: The result below is collected under a *controlled* injector, and that is the change that makes it worth reporting. `docker kill` alone lands 368.4 ms after it is issued on this host; freezing the container first with `docker pause` and killing it afterwards lands in 58.3 ms, with a third of the spread, and leaves the container in the same end state. The fault class does not change — it is the same F3 crash-stop at the same instruction boundary — so the two are comparable, and the injector's own contribution to the race is narrowed by measurement rather than argued away.

Over 3 sessions and all 3 capability classes, AEP-full's unwanted-applied rate is 0.0112 (3 of 269 executions) and B3's is 0.9778 (264/270), on the same injected fault. The between-session spread in AEP-full's count — the quantity a single collection cannot see, and the one the earlier version of this result was criticised for not reporting — collapses to 1 count per class across the 3 sessions. B3 stays at its ceiling throughout, never below 29 of 30 in any class in any session, which is what rules out the reading that the injector simply disabled both arms.

b) The result fell below its own pre-registered bound, and we report that as a surprise rather than a success: The prediction was registered before the data existed: 0.058, bounded [0.045, 0.081], derived from the measured landing distribution as the fraction of a $1\,000$ ms `WAITAOF` window the injector leaves open. The observed 0.0112 is *below* that bound. The pre-registration fixed in advance what a rate outside the bound would mean — the protocol behaving unexpectedly, not the mechanism succeeding — and we honour that here rather than presenting a lower-than-predicted number as a better result.

The reason the bound was wrong is measurable, and it is not calibration: the model behind it fails on two independent premises that push in opposite directions. First, it treats the exposure window as the injector's bench-measured landing latency. That does not survive contact with a collection: 106 of 240 uncontrolled AEP-full runs received a genuine `WAITAOF` acknowledgement *after* the bench landing had supposedly passed, and a `WAITAOF` acknowledgement requires a live server. Timed during the collections themselves, the same injector call takes 1 038.5 ms against the bench's 368.3 ms — 2.8×, with ranges that do not overlap. Second, it treats the `WAITAOF` wait as uniform on $U(0, 1\,000)$ ms. Measured over the runs the model was calibrated against, the fraction of waits under 58.3 ms is 0.0042, where uniformity assigns 0.0580 — 13.9× too generous, and worse the narrower the window. The two errors partly cancelled at the uncontrolled window and could not at the controlled one.

The calibration that appeared to license the bound compared the wrong two quantities. The model predicts *dispatch*; the estimand is an *applied effect*, and dispatch is necessary for one without being sufficient. On the same runs the dispatch rate is 0.596 and the applied rate is 0.546. Against the quantity it actually predicts, the model was already wrong on its own calibration data. We record this because a pre-registered bound that agrees for the wrong reason is worse than one that disagrees, and only the second kind is visible without looking.

c) What this does and does not license us to say: We report a rate and its mechanism, not a guarantee. **We do not claim that AEP-full prevents the unwanted effect.** The residual is non-zero and is irreducible under this fault class: freezing Redis synchronously at the checkpoint would make `WAITAOF` unanswerable, but B3 reaches the same checkpoint and still needs two further Redis calls, so the same freeze would stop B3 dispatching too and the contrast would vanish because the injector had disabled both arms. What the controlled fault buys is that the residual is now bounded by something we measured rather than by this host's `docker` latency — and, as above, the bound we derived from that measurement was itself wrong in a way the data can show. The magnitude is not a constant of the protocol and this paper does not offer it as one.

TABLE IX
THE *uncontrolled* HARD-REDIS-KILL ABLATION — THE ORIGINAL `DOCKER KILL` CELL, RETAINED AS THE REPLICATION THAT AGREES IN DIRECTION WITH THE CONTROLLED RESULT ABOVE. IDENTICAL FAULTS; THE ONLY DIFFERENCE IS WHETHER THE SYSTEM WAITS FOR THE DURABILITY ACKNOWLEDGEMENT BEFORE DISPATCHING. THE RATE IS OVER EXECUTIONS, ONE PER RUN. SOURCE: `REDIS-KILL-ABLATION.CSV`, REGIME `REDIS-KILL-PREACK`, NO-READBACK.

	runs	unwanted count	applied effect rate	canary survived
AEP-full	30	10	0.3333	30/30
B3 (no barrier)	30	28	0.9333	30/30

d) The uncontrolled cell replicates the direction, and is not pooled with the controlled one: The original collection used `docker kill` alone, and is kept as the uncontrolled replication. It is reported separately rather than pooled: the two injectors have different landing distributions, and combining them would trade a stated limitation for an unstated confound. The scope of that cell is one pre-acknowledgement crash point and one host; the AUTHORITATIVE-READBACK class was subsequently collected over 4 sessions (Section VIII), a comparison that did not reach a precision adequate to the question.

There the rate is 0.3333 for AEP-full and 0.9333 for B3 (Fisher's exact test, two-tailed, $p = 1.9 \times 10^{-6}$), a difference of 18 executions on the same injected fault — the same direction as the controlled cell, at a magnitude the uncontrolled injector sets. Note that this is the only fault in the study under which the two systems diverge; everywhere else they differ by at most 2 executions in 540 (Table VIII). Both systems declared ambiguity on all 30 runs: the endpoint is NO-READBACK, so neither can resolve what it does not know, and both fail closed. The difference between them is not in what they *say* — that is the detection guarantee, and Section VI-C1 already showed it does not depend on the barrier — it is in what they did to the outside world before saying it.

That magnitude moves, which is why the controlled cell exists. The uncontrolled cell was re-collected 4 further times under an identical configuration — same cell identity hash, same per-run seed, same pinned Redis image — for 120 executions per arm. AEP-full's unwanted-applied count ranged 4–20 out of 30; B3 recorded 28/30 in every one, a range of 0. Taking the session rather than the execution as the unit, the difference averaged 17.2 effects, 95% interval $[6.1, 28.4]$, and the 18 of Table IX falls inside it. Against that spread, the controlled cell's 1-count per-class spread over 3 sessions is the change worth reporting.

The variation has a candidate cause — the race itself — and the replication set cannot establish it. Taking the session as the unit, the four sessions separate the runs that applied an effect from those that did not by +70, -4, +282, +74 ms: mean +105 ms, 95% interval [-91, +302]. **The half-width is 196 ms, 1.86 times the mean it brackets**, and the interval contains zero. We give the four session differences individually because the summary is what fails here: the interval admits effects three times the mean and effects of the opposite sign, and this design does not distinguish them. We give no p, and we do not put a count of sessions on one side of zero in place of one. Over 4 sessions a two-sided sign test cannot return below 0.125, so no outcome this design admits could have reached significance.

B3 is not the negative control this paper previously reported. Pooled over the same 120 executions it read as a clean null, and pooling is what made it read that way. At the session level it is uninformative: -28, +27, -31, -152 ms, mean -46 ms, interval [-166, +74], a half-width of 120 ms and 2.61 times its own mean. A comparison that wide could not have contradicted the mechanism whatever B3 had done, so its failure to contradict it was never evidence for it. The original cell was collected on a different filesystem from the four replications, a difference no run configuration records, so we report it separately: it agrees in direction at 88 ms over 30 runs ($p = 0.03$), with its own control at 26 ms ($p = 0.53$, $n = 30$). **It is one session: it pools nothing, and it replicates nothing.**

e) Where the fault lands after the branch point, the arms tie: The same fault delivered *after* dispatch separates nothing, and we collected that rather than inferring it. Arming the kill at the last instruction before transmission and delivering it 200 ms later puts it in flight, after both arms have already dispatched. Over 2 sessions the two arms differ by at most 1 count in any capability class, and both sit at their ceiling. Two qualifications belong with that number. The second session is a deterministic replay of the first rather than an independent draw — the harness assigns seeds per cell and repetition, so where no race remains the second collection reproduces the first exactly, which is itself the clearest evidence that no race remains. And B3's lowest cell is 27/30, exactly the floor below which the result would have been read as a mis-timed injector rather than a tie; it is the boundary, not headroom.

What is structural, across all three faults, is that the race exists for AEP-full at all and cannot exist for B3.

3) The barrier's durability claim, under the fault that can actually test it: There remains a third thing the barrier is supposed to do, and it is the one most readers assume it does first: protect the write-ahead record from being lost when Redis dies. We tested that premise before building on it, and for the fault everyone reaches for first, it is false.

10 trials, each phase-aligning the `everysec` timer with a `WAITAOF` so the window under test is as wide as it can be, writing a key *without* waiting for it, hard-killing the container with `docker kill -s KILL`, restarting, and reading back. Every kill landed (every container came back with its `uptime` at zero) and every kill landed inside the window (write-to-death 419–992 ms against a 1 000 ms fsync period). **Not one unfsynced write was lost:** $0/10$**.**

The mechanism is not subtle once seen. `appendfsync everysec` defers the `fsync(2)`, **not** the `write(2)`: Redis writes the AOF buffer to the operating system on every event-loop iteration. A `SIGKILL` destroys the *process*; the kernel and its page cache are untouched, and a kernel that is still running flushes those bytes to disk. The same result reappears at n=60 inside the ablation cells above, where a canary write made immediately before each kill survived in 30/30 and 30/30 runs.

That narrows the claim to a named fault class — loss of the page cache: host power failure, kernel panic, VM destruction — and an earlier draft of this paper stopped there, naming a fault it had not injected. We now inject it.

a) Making the storage lose the writes: The probe is the one above with the fault swapped. A loop-backed block device carries a `dm-flakey` target whose `drop_writes` feature silently discards every write bio while serving reads correctly, which is what a storage stack looks like to a kernel whose writes will never reach the platter. Redis 7.2.5 — the same build as the rest of the evaluation — runs with its append-only file on an ext4 filesystem over that device. Each trial writes one key through `WAITAOF` and one without, switches the device into `drop_writes`, kills the server, unmounts to discard the page cache, restores the device and reads both keys back. The switch is the instant of power loss: what was `fsync`-ed before it is on the device, what was still dirty is not.

Two details decide whether this measures anything. `dmsetup suspend` freezes the filesystem and flushes outstanding I/O by default, either of which would push the unacknowledged write out before the fault took effect and produce a confident null; the reload runs with `--nolockfs --noflush`. And a self-test runs first on the device alone, with no Redis in the picture: a file `fsync`-ed before the switch must survive and one `fsync`-ed after it must not. The probe refuses to report Redis trials unless that holds.

b) The result: The probe ran 90 trials in 3 independent replications. Each replication builds the device stack from scratch, and each *trial* within it rebuilds the filesystem with `mkfs.ext4` and starts a fresh server, so no trial can read a key a previous trial's append-only file still holds. **Separation was perfect in every replication**: the acknowledged record survived 30/30 and the unacknowledged one was destroyed 30/30 in each, 90/90 and 90/90 overall. Against the same two keys under a process kill, $0/10$ were lost; the fault classes separate completely.

We give no p for either comparison, and the reason is that no unit supports one: at the trial level the test would assume an independence the shared device stack violates and would ignore the pairing — both records are written in the same trial and meet the same write-loss event — while at the replication level, three against three, the two-sided minimum a perfectly separated result can reach is 0.1, which is the design's floor rather than a statement about these data. The complete separation in every replication is the finding, and it does not need one. The filesystem was mounted `ext4 rw,relatime` with its journal enabled and no `nobarrier`, and the server's `appendfsync` was read back as `everysec` from the running instance; the probe refuses to measure if either is otherwise, because under `always` there is no unacknowledged write to lose and under `nobarrier` a surviving acknowledged one would prove nothing.

The barrier's durability benefit is therefore real and now measured, and the fault class matters more than the mechanism. The Redis documentation's "you may lose

TABLE X
MEDIAN STEP LATENCY, CRASH-FREE RUNS ONLY, E5-GATED. INCREMENTS ARE OVER B0, THE NO-PROTOCOL SYSTEM ON THE SAME HOST AND PROVIDER. THE WHOLE WRITE-AHEAD PROTOCOL MINUS THE BARRIER IS THE B3 ROW; THE BARRIER IS THE DIFFERENCE BETWEEN B3 AND AEP-FULL.

system	runs	median step (ms)	over B0 (ms)
B0 no protocol	3	2 010.2	—
B1 + lease	3	2 014.0	3.9
B2 + fenced CAS	3	2 017.4	7.3
B3 full protocol, no barrier	3	2 038.2	28.0
AEP-full	3	4 004.9	1 994.7
B4b durable, 1 attempt	3	4 013.4	2 003.3
B4 durable, ∞ attempts	3	4 015.7	2 005.5

1 second of data if there is a disaster" [8] is correct; what our two probes add is that "disaster" must mean the host, that the widely repeated reading of the sentence as covering a crashed Redis *process* is wrong, and that once the host is the fault the sentence is exactly right.

One qualification, and it is the same one the process-kill probe carries: the exposure window is deliberately maximised. The unacknowledged write is issued 12.4–61.8 ms before the device stops accepting writes, inside a 1 000 ms fsync period that a preceding `WAITAOF` has just reset. 90/90 is therefore the loss probability for a power cut placed where it does the most damage, not the rate a uniformly-timed one would produce; that rate is bounded above by it and approaches it as the write approaches the start of the period. The design is deliberate and it is the same design that made the process-kill probe's $0/10$ strong: both probes give the loss hypothesis its best case, and only one of them finds loss.

D. RQ3 — cost, and how much of it is optional

Only crash-free runs on a suspend-disabled host contribute. The provider is configured with a constant 2 000 ms delay, so one round trip is the floor and the interesting quantity is the increment over it.

The overhead is the barrier, and nothing else. A median step costs 4 004.9 ms under AEP-full against 2 010.2 ms under B0, which does nothing but call the provider. Almost all of that gap is one mechanism. Everything the write-ahead protocol does apart from waiting for `fsync` — the intent ledger, the vault, the request binding, the preflight, the recovery service — costs 28.0 ms on a two-second call, or 1.4%. The two `WAITAOF` round trips cost 1 966.7 ms between them, or 983.3 ms each.

Read against Section VI-C1 that decomposition says something stronger than "AEP costs two seconds". The 28.0 ms is what the detection guarantee costs, because 28.0 ms is the difference between B0 and B3 and B3 is the system that already has the detection guarantee in full. The 1 966.7 ms buys the prevention guarantee of Section VI-C2 and nothing else.

That figure is also a property of a *configuration*, not of AEP. Under `appendfsync everysec` an acknowledgement cannot return until the next scheduled fsync, and those are one second apart, so a barrier costs up to a second and the protocol pays that twice. What we measure is 983.3 ms per barrier, which sits near the top of that range rather than in the middle of it; we did not measure why, and we do not claim a mechanism for it here.

1) The barrier is a deployment choice, with three measured points: The $\approx$ 2-second figure above is the one a reader would carry away as "AEP's overhead". It is not: it is the price of one of the protocol's two guarantees, under one of the durability settings that guarantee can be bought at. Table XI is the decision as a deployment actually faces it.

Three points, and the third is the one the earlier framing of this paper concealed.

a) `everysec` with the barrier: The default, and the expensive one. An acknowledgement must wait for the next scheduled fsync, up to a second away, and the protocol waits twice.

b) `always` with the barrier: The identical crash-free cells — same systems, endpoint, regime, seeds, host and provider — against a Redis whose only changed line is `appendfsync always`. The barrier's cost falls from 1 966.7 ms to 15.0 ms, because under `always` the fsync has already happened by the time the write returns, so `WAITAOF` has nothing left to wait for.

Both figures are ablation differences *within* one policy, which required collecting the barrier-less arm twice. It would have been easier to subtract the `everysec` B3 median from the `always` AEP-full median, and that would have assumed the ablated protocol's own writes cost the same under both policies. They nearly do — B3 is 2 038.2 ms under `everysec` and 2 048.4 ms under `always` — but "nearly" is a measurement, and an order-of-magnitude claim resting on an unmeasured premise is the kind of number this paper is trying not to publish.

c) How precise these two figures are, which is: not very: Each is a difference of two medians over three runs per arm, and we report a cluster bootstrap over runs rather than over executions, because thirty step latencies from three runs are not thirty independent draws. The intervals are wide and one of them is uninformative:

`appendfsync`	barrier (ms)	95% CI
`everysec`	1 966.7	[477.9, 1 978.8]
`always`	15.0	[-1 474.6, 22.7]

Under `everysec` the barrier's cost is comfortably positive and its magnitude is pinned only to within about a factor of four. Under `always` it is *not separable from zero at this sample size* — the interval spans zero and is dominated by a heavy upper tail that three clusters cannot resolve. So the honest form of the claim is directional and bounded, not a ratio: **under `everysec` the barrier costs hundreds to $\approx$ 2 000 ms per step, and under `always` we cannot show it costs anything at all.** That is enough for the deployment decision in this section, and it is less than the point estimates alone appear to say. Closing it needs more crash-free runs, not more executions per run, and Section VIII says so.

d) B3-mode: the barrier removed entirely: This row is not a baseline we are beating; it is a supported configuration of

TABLE XI

THE BARRIER IS A DEPLOYMENT CHOICE, NOT A FIXED COST. ALL THREE ROWS RUN THE SAME PRE-DISPATCH INTENT LEDGER AND THEREFORE MAKE THE SAME DETECTION CLAIM; THEY DIFFER IN WHAT THEY PAY TO ALSO WITHHOLD DISPATCH WHEN DURABILITY CANNOT BE CONFIRMED. THE BARRIER COLUMN IS EACH ROW'S OWN MEDIAN MINUS A B3 MEDIAN COLLECTED UNDER THE SAME `APPENDFSYNC` POLICY. CRASH-FREE RUNS ONLY, E5-GATED, 2 000 MS PROVIDER FLOOR.

configuration	median (ms)	over floor	barrier (ms)	prevents	claim
AEP-full, `everysec`	4 004.9	2 004.9	1 966.7	yes	detection + prevention
AEP-full, `always`	2 063.4	63.4	15.0	yes	detection + prevention
B3-mode, `everysec`	2 038.2	38.2	0.0	no	detection only

The detection claim of Table VI shows no observed difference in any cell, bounded by pooling the capability classes rather than per class (Section VI-C1): it is produced by the pre-dispatch record plus no re-entry, which all three rows have, and Table VIII is the ablation that shows it. What the barrier buys is the last column's second word, and Table IX is what it is worth. 'Over floor' is the same median less the provider's 2 000 ms delay, and so includes the 28.0 ms the protocol costs with the barrier already removed.

the same implementation, and Section VI-C1 is what licenses putting it in this table. It pays 28.0 ms over a system with no protocol at all. On the detection metrics, pooled across capability classes, we observe no difference between it and AEP-full, and what supports that is a bound rather than a test: both the undetected-duplicate and lost-effect rates lie in $[0, 4.77\%]$ at joint coverage of at least 90%, and the declared-ambiguity difference lies inside the ± 5 percentage-point margin that Section VI-C1 states as a post hoc stipulation rather than a preregistration.

We do not make that claim per capability class, and the reason is a number rather than an oversight. Applying the same construction at per-class scope means six simultaneous bounds instead of two, over a third as many run clusters, and Bonferroni then puts the width at 20.1 percentage points. A band that wide is not a detection guarantee, so what we report is that these data cannot support a per-class equivalence claim worth stating — not that the question does not arise. What it forfeits is Section VI-C2: under a hard Redis kill it will put an effect on the wire that AEP-full withholds, at 0.9333 against 0.3333, and its intent record will not survive a host-level write loss (Section VI-C3).

So the question a deployment answers is not "can we afford AEP" but "do we need prevention, and at which fsync policy". If losing Redis is the fault to worry about — an instance on the same host as the workers, an unreplicated instance, a VM that can vanish — the barrier is the point of the protocol and `always` makes it nearly free at this workload's shape. If Redis is well protected and worker crashes are the fault of concern, B3-mode gives the paper's headline result for 28.0 ms.

One measurement this table does not support. It says nothing about what `always` costs under load, and we are not recommending it on this evidence. The Redis documentation calls `always` "very very slow, very safe" [8], and that judgement is about write throughput, which our workload cannot exercise: two workers, a two-second provider delay, and a handful of writes per step leave the system latency-bound and nowhere near an fsync-rate limit. Our throughput figure is in fact *higher* under `always` (0.42 versus 0.33 executions/s), which is a statement about this workload's shape and not about Redis. A write-heavy deployment would land somewhere else on the curve entirely, and finding where is not something this evaluation attempted.

E. RQ4 — recovery

Recovery behaviour is reported through the outcome distribution rather than a success rate, for a reason worth stating: "recovery success" means two different things either side of whether a recovery service was running. Where one ran, a crashed execution reaching a terminal classification was *recovered*. Where none ran — B0, B1, B2, B4, B4b declare none — the same execution reached its classification because a supervisor ran the step again, which is not recovery but the thing recovery exists to avoid. Our analysis excludes runs without a recovery service from that metric rather than scoring them zero or one, because both would be statements about a component that was not present.

Within that scope: every AEP-full and B3 execution in the crashed regime reached a terminal classification, and across 432 runs exactly one recorded a reconciliation disagreement — every other completed run's event log agreed with its independently written oracle ledger. The harness performs this reconciliation after each completed run; it checks definitive applied/non-applied assertions, effect-count bounds, attribution, and duplicate predictions rather than trusting a line count.

That one is worth stating precisely, because the check exists to be believed. It was a B4b cell in which the system classified all ten executions CONFIRMED-APPLIED while the provider's ledger recorded two, eight disagreements of the form "the system asserted the effect was applied and the ledger records none". Both sibling runs of the same cell recorded ten, the provider logged no error, and its run-log held three lines against eleven for each sibling — so the provider recorded almost nothing while answering the workers successfully. We could not determine the cause. A re-run of the identical cell, same seed and same configuration, agreed.

We treat it as a *void* run on the rule the durability probe uses: a trial whose measurement precondition failed says nothing about the system in either direction. Counting it would let a broken ground truth set a baseline's rate; discarding it quietly and keeping the re-run would be re-running until the number looks right. So the re-run is what Table VI contains, the voided attempt must ship in the external raw archive under `results/voided/` with this explanation beside it; it is not present in the current Git repository. The count above is one rather than zero. Recovery latency is not reported as an absolute figure: the E5 gate leaves too few gated crashed runs to support a distribution, and Section VIII records this as a gap rather than filling it with ungated numbers.

VII. Related Work

AEP composes established mechanisms: leases, fenced compare-and-swap, write-ahead logging, and an fsync barrier. We do not claim that durable intent plus locking is new. The contribution is a capability-specific recovery semantics for effects outside the coordinator's atomicity domain, and an evaluation that separates detection from prevention.

a) Leases and fencing: Leases provide time-bounded mutual exclusion under crash faults [9]; Chubby made the pattern an infrastructural service [12], and the Redis lock pattern [13] is a commodity form. A lease holder may stall past expiry and write afterwards, so a monotonic fencing token is also required [11]. AEP applies that standard remedy to its state store. B1 and B2 show its boundary: fencing protects the store, not an external endpoint that does not interpret the token (Table VI).

b) Logging, outboxes, and long-lived transactions: The expected-version CAS is optimistic concurrency control [14]. ARIES writes intent before an action that the same recovery domain can undo or redo [15]; AEP instead records an action performed by a foreign endpoint that may support neither operation. The transactional-outbox pattern likewise stores a message before a relay publishes it, but explicitly permits the relay to publish twice after a crash and therefore assumes an idempotent consumer [16]. B3 is structurally outbox-like, with a no-re-entry rule in the measured runner; it does not make the remote mutation transactional. Sagas are appropriate when a compensating action exists [17]. AEP addresses the case in which no compensation exists, or deciding whether to compensate first requires the missing outcome.

The expired IETF HTTPAPI working-group draft for an `Idempotency-Key` header documents the cooperative alternative: a server can make non-idempotent `POST` and `PATCH` requests safely retryable by interpreting a client key [18]. AEP's premise is precisely the endpoint that does not implement such a contract. Helland describes the broader consequences of operating without distributed transactions and the centrality of idempotence [1], [19].

c) Locks with intent and durable execution: Olive is the closest mechanistic antecedent. Its locks with intent durably associate code with a lock and provide exactly-once execution despite client failures and concurrent duplicate executions [20]. A write-ahead intent as a checked precondition of execution is therefore prior work, not AEP's novelty. Olive's guarantee is realized through operations made recoverable within its cloud storage abstraction. AEP starts where that atomicity scope ends: the effect is performed by a legacy endpoint, and in the NO-READBACK class no participant can later establish whether it happened. AEP consequently offers no exactly-once claim; it terminates in declared ambiguity and prohibits automatic re-dispatch.

Beldi [21], Durable Functions [2], Boki [22], and ExoFlow [3] develop transactional or replay-based durable execution. Their guarantees are appropriate when an effect is idempotent, transactional, compensable, or otherwise recoverable. Temporal states the operational premise directly: activities should be idempotent because they may execute more than once [4]; setting Maximum Attempts to one selects the opposite retry policy [7]. Our B4 and B4b controls instantiate these two policies against the same non-cooperative endpoint. They are semantic controls, not claims about those products' complete implementations.

The ablation supplies AEP's narrower empirical delta. B3 and AEP-full share the pre-dispatch record and no-re-entry transition but differ on the durability barrier. We observe no difference in their crashed-regime detection outcomes, bounded by pooling the capability classes: both zero-event rates fall within $[0, 4.77\%]$ at joint coverage of at least 90% (Section VI-C1). The Redis-kill experiment isolates a prevention effect. We found no prior evaluation that measures this detection/prevention decomposition with an independent external-effect ledger.

d) Agent execution reliability: Three contemporaneous systems overlap substantially and delimit the claim. Verified Tool Calls formalizes non-atomic agent tool failures and uses postcondition verification, verify-before-retry, and idempotency keys in a simulated fault environment [23]. Its verifier requires a queryable postcondition; without server-side idempotency, it characterizes the remaining race as best-effort. AEP evaluates the authoritative, positive-only, and absent-readback cases separately and uses terminal ambiguity for the last case.

LogAct durably exposes intentions in a shared log before execution and supports recovery and policy voting for open-ended agent control flow [24]. It is broader in agent-state-machine structure and isolation; AEP is narrower in studying what can be concluded about one non-idempotent external mutation after an interruption. The Sovereign Execution Broker (SEB) reserves certificate nonces before mutation, disallows retry after possible invocation unless durable or target-side evidence proves otherwise, and provides a mandatory broker boundary with cryptographic certificates [25]. That boundary is stronger than AEP's trusted-process guard. AEP's distinct evidence is the three endpoint-capability classes, terminal declared ambiguity, real `SIGKILL` and Redis durability faults, and the B3/AEP ablation. ACRFence addresses semantic rollback during agent checkpoint-restore [26]; it is adjacent rather than a basis for ranking which work is "nearest."

e) Fault-injection lineage: AEP's named crash boundaries and independent oracle follow a mature crash-consistency methodology. ALICE constructs possible post-crash storage states and checks application invariants [27]; Torturing Databases combines controlled power-fault injection with record/replay and cross-layer diagnosis [28]; and CrashMonkey/Ace exhaustively test bounded workloads at persistence boundaries [29]. AEP applies this lineage to an agent-to-endpoint protocol: real process kills at named instruction boundaries, a separately written provider ledger, and a `dm-flakey` write-loss probe. The harness is an evaluation instrument, not a claimed fault-injection contribution.

f) Revised novelty position: AEP's claim is not the invention of write-ahead intent, locking, or fail-closed interposition. It is the measured composition for endpoints with authoritative, positive-only, or no readback: capability-specific recovery; a terminal declared-ambiguity outcome when no query can settle the effect; real process and durability-fault evaluation with an independent ground-truth ledger; and an ablation that assigns detection to the pre-dispatch record plus no re-entry, and prevention to the durability barrier.

VIII. Threats to Validity

A. Construct validity

a) The barrier's durability benefit is measured on an emulated fault, not a real power cut: An earlier draft of this paper named the fault class `WAITAOF` defends against — loss of the page cache — and did not inject it, which left the durability half of the barrier's case resting on an argument. Section VI-C3 now injects it with a `dm-flakey` target in `drop_writes` mode, and the result is unambiguous ($90/90$ acknowledged records survive, $90/90$ unacknowledged ones do not).

What remains is that `drop_writes` is an *emulation* of a power cut, not a power cut. It is faithful in the dimension that matters — writes issued after a chosen instant never reach the platter, while everything `fsync`-ed before it does — and unfaithful in others: a real power loss can tear a sector mid-write, can lose writes the device claimed to have committed to its own volatile cache, and does not politely stop at a boundary we chose. Those differences all make real hardware *worse* than our emulation, so the direction of the result is safe; its precision is not. Specifically, we cannot claim the acknowledged record would survive a real power cut on a device with a lying write cache, because `fsync` on such a device does not mean what the barrier assumes it means. That is a property of the storage, not of the protocol, and AEP inherits it from every system that trusts `fsync`.

We also note what the probe deliberately does not model: the exposure window is maximised by phase-aligning to a just-completed fsync, so $90/90$ is the loss rate for a worst-placed cut rather than a uniformly-timed one (Section VI-C3).

b) The write-loss probe tests `WAITAOF`, not AEP: Section VI-C3 runs two Redis keys against a device that stops accepting writes. It does not run AEP-full and B3 through the evaluation harness under that fault, and so it does not measure a difference in *protocol* outcomes — duplicates, lost effects, declared ambiguity — attributable to write loss. What it establishes is the premise the protocol's durability argument rests on: that an acknowledged record survives an event that destroys an unacknowledged one, which under a process kill is false and under write loss is true. Extending it to the full systems needs the harness's Redis to run on the flakey device, and the harness's Redis is a container whose bind mounts this Docker resolves in the Windows filesystem (Section VIII, platform), so the two do not currently compose. We regard that as the most worthwhile single extension to this evaluation and we have not done it.

c) The write-loss probe runs several layers above hardware, and does not need to be closer: Our `dm-flakey` target sits on a loop device backed by a file on the WSL2 root filesystem, which is itself a virtual disk on the host. A reviewer is entitled to ask what a "block device" means five layers up. The answer is that the layering is irrelevant to the claim, because `dm-flakey` *is* the boundary under test: the experiment asks whether a `WAITAOF` acknowledgement corresponds to bytes that have crossed a chosen point in the storage path before that point stops accepting writes, and everything below the target is on the far side of it in both arms equally. What the layering *would* threaten is a claim about absolute fsync latency on real hardware, which we do not make from this probe — the barrier's cost comes from Table XI, measured elsewhere. It would also threaten a claim that a real power cut cannot do worse, which we explicitly do not make above.

d) No process-level fault can exercise the barrier's durability at all: Stated separately because it is the more surprising half and it is the half a reader is most likely to have assumed the other way. `appendfsync everysec` defers the `fsync(2)` and not the `write(2)`, so a `SIGKILL` of Redis loses nothing that `appendonly yes` was not already keeping ($0/10$, and n=60 canaries in the ablation cells). The consequence for the claims is explicit: **under a process fault the barrier's contribution is prevention (Section VI-C2, $p = 1.9 \times 10^{-6}$, replicated at $[6.1, 28.4]$ effects prevented) and not durability**, and any reading of the barrier as protecting the record from a crashed Redis process is wrong.

e) The decomposition weakens our own novelty claim, and we would rather say so than let a reader find it: Section VI-C1 attributes the detection guarantee to the pre-dispatch record and the transition table's missing re-entry edge, and not to the barrier. Contribution C2 is the barrier machinery: an fsync acknowledgement returning an opaque, non-copyable, single-use, scope-bound in-process dispatch guard that the pre-dispatch script refuses to proceed without. Put those together and the uncomfortable reading is available: our most novel mechanism serves the claim with the *narrowest* evidence — one capability class, one crash point, and an effect size whose host-dependence we suspect and have not established — while the claim with the strongest evidence rests on write-ahead logging, which is decades old, plus a state-machine property that is a few lines of Lua. Narrowest is not weakest: within that scope the effect replicated across 4 further sessions at $[6.1, 28.4]$ effects prevented, the only session-clustered interval in this paper that excludes zero.

The cell is no longer collected only once: Section VI-C2 reports 4 further collections under an identical configuration, and they sharpen the limitation rather than removing it. The *direction* held in all five — B3 committed effects AEP-full withheld every time, and B3's own count did not move by a single execution — but the *magnitude* ranged 4–20 applied effects out of 30. The protocol's own logic offers a reason — whether AEP-full dispatches is decided by whether `WAITAOF` returns before the kill lands, so the effect size should be a function of the host's kill-latency distribution — but our measurement of that reason does not establish it. The four

TABLE XII
SELECTED DESIGN ASSUMPTIONS. "ENDPOINT MUST COOPERATE" DENOTES IDEMPOTENCE, AN IDEMPOTENCY KEY, TRANSACTION ENROLMENT, OR COMPENSATION. THE ROWS ARE NOT ORDERED BY GUARANTEE STRENGTH; THEY EXPOSE DIFFERENT RESIDUALS WHEN THEIR STATED ASSUMPTIONS FAIL.

	endpoint must cooperate	can report *unknown*	residual on failure
Lease / fenced CAS [9], [11]	n/a	no	duplicate effect
Sagas [17]	compensation	no	compensation gap
Beldi [21]	yes	no	—
Durable Functions [2]	yes	no	duplicate or loss
Boki [22]	yes	no	—
ExoFlow [3]	yes	no	—
Temporal-style, ∞ retries [7]	yes	no	duplicate effect
Temporal-style, 1 attempt [7]	yes	no	lost effect
AEP	**no**	**yes**	**declared ambiguity**

sessions separate the two outcomes by +70, -4, +282, +74 ms, mean +105 ms, 95% interval [-91, +302], a half-width 1.86 times that mean: an interval that contains zero and is wider than the effect it brackets, because the sessions disagree. A reader should therefore treat 18 as one draw from that distribution and not as a property of AEP. A second host would not replicate the number; it would sample a different distribution, and the prediction this paper is willing to make about it is directional only.

We think that reading is correct and that it is the paper's contribution rather than a hole in it. The composition being novel was never the argument; Section VII says so in its first paragraph. What is new is the *semantics* the composition delivers and, now, an ablation saying which half of the composition delivers which half of the semantics — including that the expensive half is optional. But a reviewer weighing novelty should weigh it against that, not against the impression the introduction gives if C2 and C3 are read without Section VI-C1 in between.

f) The detection finding has no referent outside this artifact: The decomposition above is established by an ablation — AEP-full against itself with the barrier removed — and an ablation is internal by construction. That is the right instrument for *attributing* an effect to a mechanism, and we would choose it again for that purpose, but it cannot corroborate the resulting claim from outside. Nothing external to this artifact establishes that a pre-dispatch record without an fsync barrier is sufficient for detection: the proposition is supported by two systems we wrote, measured by a harness we wrote, against a provider we wrote. The oracle is independent of the systems under test — rates are counted from the provider's separately written ledger and a parsed source gate prevents the analysis from reading live protocol state (Section VIII, internal validity) — which addresses whether we counted correctly, and does not address whether the finding generalises beyond our implementation of the pattern. A reviewer who distrusts the harness has no independent purchase on this result, and we know of no way to supply one short of re-implementation by someone else. That is the structural cost of the instrument, it applies to the paper's most load-bearing empirical claim, and we prefer to state it here than to let the ablation's internal consistency read as external support.

g) The dispatch guard assumes trusted same-process code: The guard prevents accidental or ordinary control-flow bypass through the supported dispatch API, and its registry detects copying attempts, reuse, and scope mismatch. It is not a cryptographic capability. Python's underscore naming, object-construction checks, and closure placement are not security boundaries: malicious or arbitrary code running in the same interpreter can reach module internals, request a guard without executing the barrier, or bypass the API altogether. The implementation therefore enforces a protocol invariant only under a trusted-code assumption. A hostile-extension threat model would require a separately enforced process or service boundary, which this artifact does not provide.

h) The mock provider's realism: The endpoint is a purpose-built service with a configurable delay, timeout and error distribution, and a transactionally written ground-truth ledger. It is realistic in the dimension the experiment turns on — its reconciliation capability, which is the independent variable of Table VI — and unrealistic in most others: no authentication, no rate limits, no partial writes, no schema evolution, one mutation shape. A real legacy endpoint may also *misdeclare* its capability, which the protocol cannot detect (Section III-C); we validate the capability's shape, never its truthfulness.

i) B4 is not Temporal: B4 shares exactly one mechanism with a production durable-execution engine: durable history, replay, and at-least-once activity re-execution. It does not model heartbeating, backoff timing, task queues, versioning, or determinism checking. Its re-execution policy is the vendor's documented default and is quoted as such [7], but the decision is our code. No claim about Temporal's throughput, latency, maturity or correctness as a product is made or supported anywhere in this paper, and B4's latency in particular should not be read as any engine's recovery latency. B4 and B4b *bracket* the configuration space rather than survey it: a deployment could set Maximum Attempts to 3, add a heartbeat, or make the activity idempotent — the first two land between our two points on the same duplicate/loss trade, and the third leaves the problem this paper is about.

j) Declared ambiguity is not evaluated as an operational outcome: The paper measures how often AEP declares ambiguity and shows that it never substitutes a silent failure for it. It does *not* show that declaring is better in practice. That would require an operator study — do teams resolve declared ambiguities, how long does it take, what fraction are resolved correctly, and does the queue stay bounded? — and we have run none. The argument that a declared incident beats an undiscovered one is a normative claim about failure handling, not a result of this evaluation, and we mark it as such. Relatedly, the artifact has no escalation *mechanism* (Table IV): reaching the terminal state pauses the execution and alerts nobody.

k) The motivating traces come from our own harness: Section II reproduces failures in baselines we implemented, not in deployed systems. It establishes that the failure modes are reachable under the stated fault model; it is not evidence about their frequency in production agent deployments, and we have no such evidence. A field study of how often agent frameworks meet non-idempotent endpoints without idempotency keys would strengthen the motivation considerably and does not exist here.

l) One author implemented every system under comparison: B0–B2 are small enough to inspect, each carries a machine-readable descriptor that tests check against its implementation, and B4's re-execution policy is defended against the vendor's own documentation in the artifact. That is mitigation, not elimination: a baseline written by the people proposing the alternative is a structural conflict, and the artifact is released partly so the baselines can be re-implemented by someone else.

m) Test counts are not protocol assurance: Stated in Section V and repeated because it is a live temptation: the suite's size is evidence about the artifact's hygiene, not about the protocol's behaviour under faults. The evidence for the latter is the fault-injection matrix.

B. Internal validity

a) A resume defect that would have inflated counts: Our matrix runner supports resumption, and a run whose previous attempt was interrupted left event shards on disk that a fresh attempt could merge into its own log, double-counting executions that were never part of the run being recorded. It surfaced as a configuration-digest mismatch — the digest check did what it exists to do — but between two attempts of the same harness version the merge would have been *silent*, and the error class is "a resumed run over-counts", which for a paper measuring rates is as bad as errors get. The safeguard is now explicit: the runner discards stale event shards and any stale summary before a run writes anything, logs what it discarded, and leaves the run's *inputs* — the rendered provider config and the oracle ledger — untouched, with a test pinning that distinction. Every resumed run in the current results was collected after the fix; runs collected before it were re-analysed and the single affected run was re-collected.

b) Two further defects the matrix found that the unit suite had not: A re-executing supervisor abandoned the lease instead of waiting for it, and execution-state seeding was not idempotent. Both made a *baseline* look better than it is, so both were biasing against our own hypothesis and are fixed; we record them because the general point — that a fault-injection harness is a stronger verification tool than a unit suite — cuts both ways, and there may be defects it has not found.

c) The test suite is destructive to a running matrix, and one batch learned that the hard way: While collecting the B4/B4b cells we ran the project's own integration suite against the same Redis. The suite is deliberately destructive: it deletes the `aep:*` namespace in the database the matrix is using, and one of its durability tests `docker restarts` the matrix's Redis container by name to prove the AOF survives. Several runs died with connection errors, and the server the batch was collecting against was replaced underneath it.

Runs that *fail* are harmless — they write no summary, and the resumption logic re-runs them. The runs that *completed* during that window are the hazard, because a Redis blip makes a re-executing baseline retry, a baseline that retries more produces more duplicates, and that bias points toward our own hypothesis. They are also indistinguishable from clean runs in every artifact the analysis reads. Every B4/B4b run collected in the disturbed window was therefore discarded and re-collected on an otherwise idle host, and the server's `run_id` was recorded before and after the replacement batch to confirm it was not restarted again; none of the discarded runs contributes a number here.

Two rules follow, and the artifact enforces both rather than documenting them: the runner refuses to start beside a host-level fault injector, and it now records the server's `run_id` with every run so that a restart during collection is visible in the results rather than inferred from a connection error that happened to be logged. We report the episode because the generalisation is not "be careful": it is that a fault-injection harness and a fault-injection test suite make the same assumptions about owning the system, and running both is a measurement error that produces ordinary-looking data.

d) Ambiguity is counted from the oracle, not from self-report: A system that declared ambiguity liberally could otherwise score well by saying "I do not know" about everything. Duplicates and lost effects are counted by comparing the provider's independently written ledger against the system's records; the analysis is prevented by a parsed source gate from reading live protocol state.

e) The oracle is independent, not infallible: The provider ledger is written outside the systems under test, and every completed run is reconciled against it, but the provider and ledger still share one implementation and one host. One voided run demonstrates that this oracle can fail: its event log and ledger disagreed while sibling and repeated runs agreed. The current repository retains the aggregate record of that exclusion, but the voided raw directory must be included in the external archive before submission. A second provider implementation or an external network/storage observer would reduce this common-mode doubt; neither is present.

C. External validity

a) Single node, single trust domain: One Redis instance, no replica, no consensus. Table IV lists what this forecloses. One residual is worth repeating here because it interacts with the results: an AOF rewind can un-fence a lease, since lock keys are deliberately not barriered, and the fix is HA rather than anything local.

b) Concurrency coverage is narrow: The harness exercises crash and takeover with two workers, so stale-owner and successor paths are covered, but it does not sustain live contention among many workers for the same execution or endpoint resource. The evaluation therefore supports fencing and recovery under the measured takeover schedule, not a scalability or fairness claim for high-contention deployments. Concurrent-claim rejection is exercised by the test suite; the fault-injection matrix supplies no contention-regime evidence for P1.

c) Platform, and a development/measurement split: All measurements were collected on Ubuntu inside WSL2 on Windows 11, with Docker Desktop port forwarding in the path, on mains power with sleep and hibernation disabled. Development and part of the test suite run on the Windows host; *every* number in Section VI comes from the Linux tree against real Redis. The split is a threat and we manage it explicitly rather than eliding it: the locally runnable Windows suite and the artifact's Linux suite against real Redis are checked separately, while the Linux evidence is the one the measurements rest on. Absolute latency and throughput are platform-bound; only comparisons *between* systems sharing this platform are meaningful, which is why Table X is presented as increments over a common floor.

d) Timing hygiene, and what it cost us: The host's modern-standby setting suspended it mid-run during earlier collection, silently inflating wall-clock durations — in one case by eighteen hours. We now require both a pre-run declaration that the host cannot suspend and a per-run wall-versus-monotonic check. The gate is strict enough that *no* run collected before it existed contributes a timing number. It also costs us coverage: recovery-latency distributions are not reported (Section VI-E), because the gated crashed runs are too few, and we prefer the gap to ungated numbers.

e) Every timing number rests on three runs, and one of them does not survive its own interval: Stated plainly because the E5 gate above is what makes it true and because Table XI is otherwise the most confident-looking table in the paper. Each cell behind it is three crash-free runs of ten executions, and the four medians in that table are what the barrier's cost is computed from.

We report a cluster bootstrap over runs for both barrier figures (Section VI-D1) and the result changes what can be claimed. Under `everysec` the barrier's cost is 1 966.7 ms with a 95% interval of [477.9, 1 978.8] — positive, but pinned only to within about a factor of four. Under `always` the point estimate is 15.0 ms and the interval is [-1 474.6, 22.7], which spans zero: at three runs per arm we cannot show that the barrier costs anything under that policy. An earlier draft of this section quoted a factor between the two point estimates. That factor is not supported by these data and has been removed.

The cause is the sample, not the metric: three clusters admit only ten distinct bootstrap multisets, and the `always` cell has a heavy upper tail (its p95 is 5 067.8 ms against a 2 063.4 ms median) that three clusters cannot resolve. Closing this needs more crash-free *runs*, not more executions per run, and the crash-free cells are the shortest thing in the plan to re-run. A deployment that needs a latency budget should measure its own.

f) Coverage of the matrix is partial, and the gaps are named: The full plan is 1 068 runs; we collected 432, comprising 3 780 executions over 126 cells, chosen to answer the research questions rather than to fill the grid. What that leaves uncovered, precisely:

- **B4's and B4b's AUTH cells were incomplete and are now collected — and completing them corrected the value.** In the previous draft B4's AUTH cell held twenty executions and B4b had none, so that cell printed as a dash. Both are now $n = 180$ and $n = 180$ over all six crash points, the same shape as every other cell in Table VI, and the whole table is free of dashes. We record what the completion changed, because it is a caution about partial cells generally and not only about this one: the earlier twenty executions came from a *single* crash point, so the figure that draft quoted was not a wide interval around the value the full cell reports — it was one crash point standing in for six, and the six differ enough that none of them represents the pooled rate. The full cell puts B4's AUTH duplicate rate at 0.5278, in line with its other two classes and far from the figure the partial cell suggested. The direction of the argument is unchanged, which is why we could describe the gap as non-essential before collecting it, but the number we quoted for it was wrong, and a partial cell being unrepresentative rather than merely imprecise is the more general lesson. Every other cell in Table VI spans all the crash points that apply to its system: six for the systems that run `aep_core`'s workflow, and five for B0–B2, which have no window between writing a record and acknowledging it durable because they write no record (Table III).
- **The prevention result rests on one cell.** The hard-Redis-kill ablation of Section VI-C2 was collected on NO-READBACK only, at one crash point, at 30 runs per system, on one host — and, as we say there, its effect size may be a function of that host's `docker kill` latency, which our measurement does not establish. It is the whole of the barrier's measured case. We predicted that the other two capability classes would move the *declared-ambiguity* column and not the applied-effect column, because whether an effect reached the provider is not something a read-back capability can change after the fact.
  We since collected those cells: 4 pre-registered sessions, 30 runs per arm per session, run-level interleaved, with the analysis fixed in advance. **The applied-effect column moved, and it moved in both directions.** The AUTHORITATIVE-READBACK minus NO-READBACK applied-rate difference ran +0.0, -10.0, +23.3 and +36.7 percentage points across the 4 sessions — a spread from

-10.0 to +36.7 points on a comparison we expected to sit at zero. That is our own prediction contradicted in our own data, and we report it rather than resolving it.
The session-clustered interval on that difference contains zero, at [-21.4, +46.4] percentage points. **It contains zero because the sessions disagree with one another, not because the effect is absent:** the half-width is 33.9 points, wider than the mean it brackets at +12.5 points, so this design could not have resolved the effect it observed. We registered a minimum detectable effect in advance and do not quote it here, because it was derived without a between-session variance component and so understates the precision this question needed; the comparison we make instead is the half-width against the observed mean. **What we report is therefore a failure to reject at a precision inadequate to the question — not evidence that capability class leaves the applied-effect column alone.** The argument above is no longer unsupported: it is contradicted in the data, and untested at adequate power.
- **The in-flight Redis-kill variant** (kill during transmission rather than before acknowledgement) is implemented and was not collected. Our own analysis predicts it ties, since neither system's barrier is in the path there — but a predicted tie that is inferred is an assumption, not evidence.
- **The** 30% **crash-probability regime** is implemented and not collected, so every rate here comes from either an all-crash or a no-crash regime and nothing in between.
- **The alternative read-back keying** is a sensitivity variant that has not been run; all results use one keying.
- **One run did not complete** and is listed as such in the manifest rather than dropped silently.

A rate we did not measure is absent from the tables rather than interpolated, and the manifest shipped with the results states the run count for every cell so a reader can see the shape of the coverage without reconstructing it.

g) One endpoint for the cost measurement: RQ3's cells were collected against a single endpoint. Overhead should not depend on an endpoint's reconciliation capability in a crash-free run, since no read-back occurs, but that is an argument and not a measurement.

D. Where the protocol should not be used

If the endpoint accepts an idempotency key, or can be enrolled in a transaction, or the effect can be made idempotent, then a durable-execution engine gives a strictly stronger guarantee at lower cost and AEP is the wrong tool. Its cost is real: two fsync barriers, which under `appendfsync everysec` dominate the protocol's latency by a factor of roughly 70 (Table X) — a ratio of medians, and one whose denominator is pinned only to [27.1, 1 524.6] ms at three runs per arm, so the factor is an estimate of an order of magnitude rather than of a figure. AEP is for the case where the precondition those systems require cannot be obtained, and its value is a declared residual rather than a better guarantee.

IX. Artifact Availability

The public repository available via the submission system contains the implementation, evaluation harness, mock provider, baseline systems, analysis pipeline, manuscript source, and the tracked derived analysis products used by the number generator. It does *not* contain the raw run directories, which are bulk data: those are carried by a separate archive, available via the submission system.

a) The raw evidence archive: 1 458 run directories across 20 collection roots, 26 300 files, 493 MB uncompressed and 24 MB compressed, covering the 432-run evaluation, the `appendfsync=always` arm, the voided oracle-disagreement run, every prevention replication, and two collections voided for having run against the wrong container runtime. Every file carries a SHA-256 in `MANIFEST.sha256`, whose own digest `87fa2d53...` attests the whole archive. Before deposit the archive was extracted, checked file-by-file against that manifest, and re-analysed: against every tracked analysis product the result was 114 byte-identical, 8 identical after two declared normalisations recorded with the deposit, and none differing.

b) What is pinned: The Python environment is locked (`uv.lock`); the Redis image is pinned by digest in the compose file; the matrix records a seed per run and emits its full plan, cost estimate and platform fingerprint *before* launching. Continuous integration provisions Redis from the same compose file the experiments use, runs the full suite, the `WAITAOF` durability suite, a citation-range check over the formal model and the numbers gate described below, and fails if any test is skipped.

c) What can be reproduced, and at what cost: The matrix is resumable and its cells are independent. The full plan is 1 068 runs at an estimated 25 h of wall time on one machine; the cells behind the results in Section VI are a subset keyed in the tracked `MANIFEST.csv`. The raw archive carries the complete per-cell and per-run manifest alongside the runs, so a reader can see what was collected rather than inferring it from the tables.

d) Regenerating the paper's numbers: The tables and every headline scalar in Section VI are generated by `scripts/paper_tables.py` from the tracked inputs enumerated in `ARTIFACT.md`: per-cell and per-execution metrics, latency under both fsync policies, the regime-labelled comparisons, the Redis-kill ablation, coverage, and the write-loss probes. Each emitted value carries a comment naming the filter and arithmetic behind it. `bash scripts/build_paper.sh` builds the manuscript and runs that check; it treats an undefined reference and a BibTeX parse error as build failures rather than warnings, because both have produced a clean-looking build of this paper once each. `scripts/check_paper_numbers.py` re-derives the tables and fails if the manuscript has drifted from the results. It also fails if a generated number is defined and never used, which is the drift a framing revision produces: LaTeX catches a macro used without a definition and is silent about a claim whose evidence was orphaned when the claim moved. The pooled summary table produced by the analysis tool is deliberately *not* a source for any number here. All comparisons used by the paper are filtered to an explicit regime; the headline and B3 comparisons use only the crashed regime.

e) The host-level write-loss probe: Section VI-C3 needs a block device that stops accepting writes, which needs root, a loop device and a `dm-flakey` target, so `experiments/flakey_write_loss.py` is not part of the test suite — a test that skips wherever those are unavailable is the failure mode this project's CI gate exists to prevent. What *is* in the suite is everything about the probe that can be checked without a kernel: that its "drop" table really carries `drop_writes`, that its pass-through table carries no feature at all, and that a trial whose acknowledged write vanished is voided rather than counted. The probe itself refuses to report any Redis trial until a self-test has shown, on the bare device, that a file `fsync`-ed before the switch survives and one `fsync`-ed after it does not.

f) The state-machine figure: Figure 1 is generated by `scripts/gen_state_machine.py`, which imports the transition set from the implementation. An edge added to or removed from the protocol changes the figure or fails the check; it cannot silently disagree with the code.

g) Citations: Every bibliography entry was verified to exist before it was written: matched against DBLP's search API, or resolved through `doi.org`, or — for vendor documentation — fetched and read on a recorded date. The verification script and bibliography metadata are tracked. The raw lookup transcript is not present in the Git repository and is not in the raw-evidence archive either, which carries run data only; it is the one item named in this section for which no external copy yet exists.

References

[1] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007, pp. 132–141.

[2] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, and C. S. Meiklejohn, "Durable functions: Semantics for stateful serverless," *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–27, 2021.

[3] S. Zhuang, S. Wang, E. Liang, Y. Cheng, and I. Stoica, "ExoFlow: A universal workflow system for exactly-once DAGs," in *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023, pp. 269–286.

[4] Temporal Technologies, "Activity definition," https://docs.temporal.io/activity-definition, 2026, accessed 2026-08-06.

[5] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.

[6] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[7] Temporal Technologies, "Retry policies," https://docs.temporal.io/encyclopedia/retry-policies, 2026, accessed 2026-08-06.

[8] Redis, "Redis persistence," https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/, 2026, accessed 2026-08-07.

[9] C. G. Gray and D. R. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP)*, 1989, pp. 202–210.

[10] Redis, "WAITAOF," https://redis.io/docs/latest/commands/waitaof/, 2026, accessed 2026-08-07.

[11] M. Kleppmann, "How to do distributed locking," https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html, 2016, accessed 2026-08-07.

[12] M. Burrows, "The Chubby lock service for loosely-coupled distributed systems," in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006, pp. 335–350.

[13] Redis, "Distributed locks with Redis," https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/, 2026, accessed 2026-08-10.

[14] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.

[15] C. Mohan, D. Haderle, B. G. Lindsay, H. Pirahesh, and P. M. Schwarz, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992.

[16] C. Richardson, "Pattern: Transactional outbox," Microservices.io, accessed 2026-08-11. [Online]. Available: https://microservices.io/patterns/data/transactional-outbox.html

[17] H. Garcia-Molina and K. Salem, "Sagas," in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.

[18] J. Jena and S. Dalal, "The Idempotency-Key HTTP header field," Internet Engineering Task Force, Internet-Draft draft-ietf-httpapi-idempotency-key-header-07, Oct. 2025, work in progress; expired 2026-04-18. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-ietf-httpapi-idempotency-key-header-07

[19] P. Helland, "Idempotence is not a medical condition," *Communications of the ACM*, vol. 55, no. 5, pp. 56–65, 2012.

[20] S. T. V. Setty, C. Su, J. R. Lorch, L. Zhou, H. Chen, P. Patel, and J. Ren, "Realizing the Fault-Tolerance promise of cloud storage using locks with intent," in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2016, pp. 501–516. [Online]. Available: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/setty

[21] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 1187–1204.

[22] Z. Jia and E. Witchel, "Boki: Stateful serverless computing with shared logs," in *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021, pp. 691–707.

[23] I. K. Mansoor, A. Phadke, and P. Rana, "Verified tool calls improve LLM agent reliability under non-atomic failures," *CoRR*, vol. abs/2608.02645, 2026.

[24] M. Balakrishnan, A. Bharambe, D. Testuggine, D. Geraghty, D. Mao, V. Venkat, I. Mironov, R. Baradi, G. Aiyer, and V. Dudin, "LogAct: Enabling agentic reliability via shared logs," *CoRR*, vol. abs/2604.07988, 2026.

[25] J. He and D. Yu, "Sovereign execution broker: Enforcing certificate-bound authority in agentic control planes," *CoRR*, vol. abs/2606.20520, 2026.

[26] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, "ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore," *CoRR*, vol. abs/2603.20625, 2026.

[27] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "All file systems are not created equal: On the complexity of crafting crash-consistent applications," in *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2014, pp. 433–448. [Online]. Available: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai

[28] M. Zheng, J. A. Tucek, D. Huang, F. Qin, M. Lillibridge, E. S. Yang, B. W. Zhao, and S. Singh, "Torturing databases for fun and profit," in *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2014, pp. 449–464. [Online]. Available: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_mai

[29] J. Mohan, A. Martinez, S. Ponnapalli, P. Raju, and V. Chidambaram, "Finding crash-consistency bugs with bounded black-box crash testing," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 33–50. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/mohan